ENGINEERING FIELD MANUAL • TECHNICAL ARCHITECTURE & SYSTEM CATALOG

Full Stack Engineering Interview Lab

A production-grade, interactive engineering simulation platform and architectural diagnosis lab built for senior software engineers, staff architects, database engineers, and engineering leadership candidates.

51	63	120	8	6	Gemini AI	Neon PG
SENIOR/STAFF SCENARIOS	TIER 1 FOUNDATIONS	SQL & POSTGRES LAB	NBFC CONCURRENCY BUGS	MONACO SANDBOXES	DUAL EVALUATION ENGINE	CONNECTED DATABASE

ARCHITECTURE & FRAMEWORK

Next.js 15.5 App Router, React 19, TypeScript 5, Tailwind CSS, Redux Toolkit

IN-BROWSER SANDBOXING

Web Worker JS Sandbox, TS Stripper, AlaSQL Relational SQL-99 Engine

AI INTEGRATION ENGINE

Official @google/genai SDK (gemini-3.5-flash / gemini-2.0-flash) + Deterministic Fallback

CONNECTED DATABASE & ORM

Neon Serverless PostgreSQL, Prisma ORM, Decoupled StorageAdapter Sync

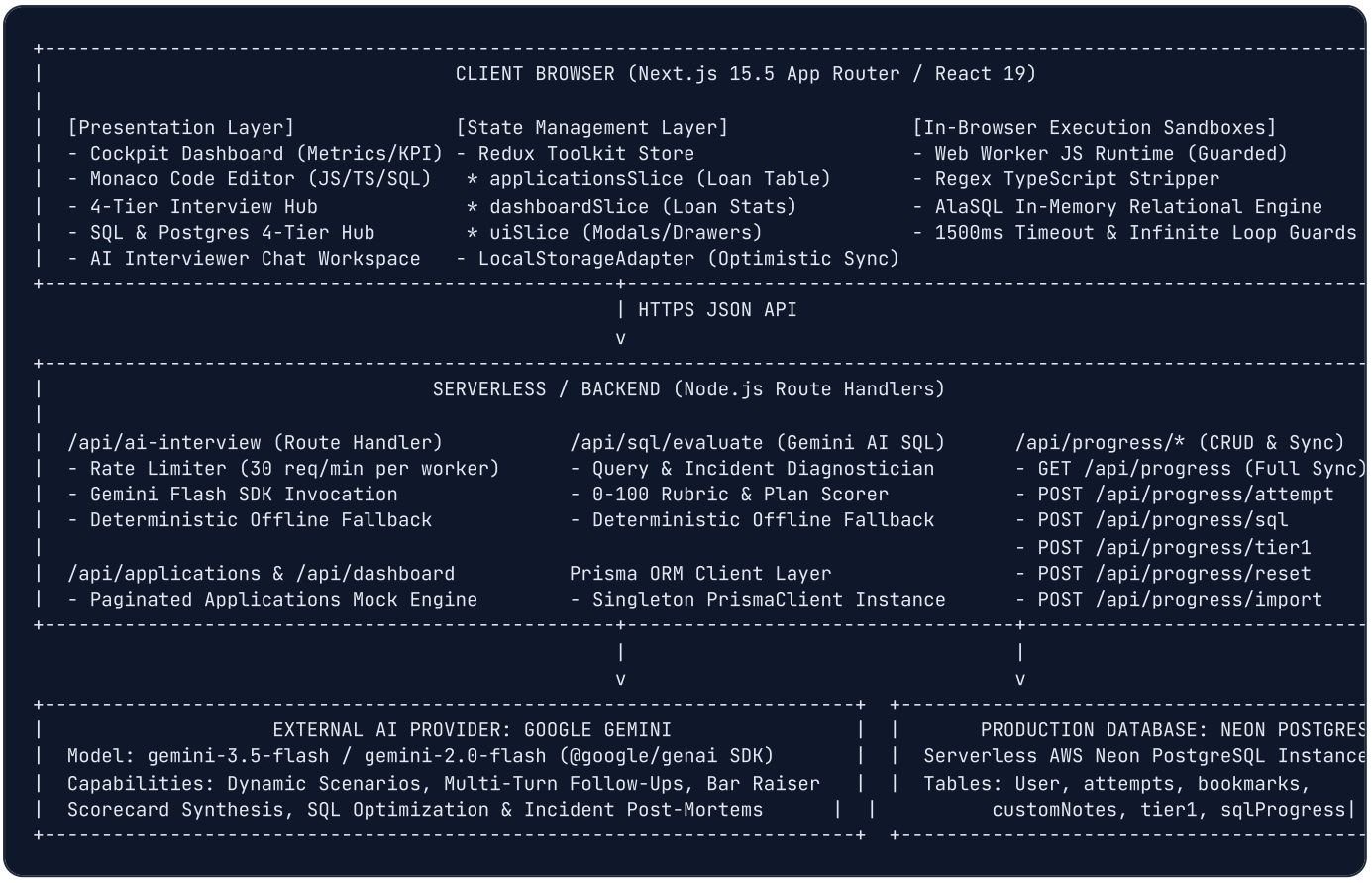
DOCUMENT VER: 2.0.0 • COMPREHENSIVE ARCHITECTURAL SPECIFICATION

CONFIDENTIAL & AUDITABLE ENGINEERING GUIDE • 248 TOTAL CHALLENGES

1. Whole System Architecture & Topology

CORE TOPOLOGY

The **Full Stack Engineering Interview Lab** is engineered as a zero-latency, high-fidelity technical interview simulator. Unlike static question banks, the platform models real-world production incident triage, distributed system architectural trade-offs, concurrency race conditions, relational database internals, and hiring committee evaluations.



1. PRESENTATION

- Next.js 15 App Router
- React 19 Server/Client
- Tailwind CSS 3.4 Design
- Monaco Editor Integration
- Lucide Engineering Icons
- Accessible Dark/Light Mesh

2. STATE & DATA

- Redux Toolkit Store
- createSlice Architecture
- Optimistic Status Updates
- Rollback Error Handlers
- Debounced Search (300ms)
- Memoized Selector Trees

3. SAFE EXECUTION

- In-Browser JS Sandbox
- Console Output Interceptor
- AlaSQL In-Memory Engine
- TypeScript Transform
- 1500ms Infinite Loop Guard
- Zero Server Remote Exec

4. PERSISTENCE

- StorageAdapter Interface
- LocalStorageAdapter (Cache)
- Prisma Neon PostgreSQL
- JSON Backup Export/Import
- Streak & Bookmark Logs
- Multi-Tier Attempt Records

Connected Production Database Architecture (Neon PostgreSQL + Prisma)

The application features a fully configured, live Neon Serverless PostgreSQL database synchronized via Prisma ORM. Persistence follows a decoupled **StorageAdapter** pattern: candidate actions (attempts, bookmarks, code submissions, custom notes, Tier 1 progress, and SQL track progress) are instantly committed to browser local

storage for zero-latency UX, while simultaneously dispatching non-blocking background synchronization requests to the PostgreSQL backend route handlers.

PostgreSQL Schema Specification (prisma/schema.prisma):
The database tracks user entities with unified JSON progress trees: `model User { id String @id @default(uuid()), email String? @unique, attempts Json @default("[]"), bookmarkedIds Json @default("[]"), notes Json @default("{}"), tier1Progress Json?, sqlProgress Json?, streak Int @default(0), lastActive DateTime @default(now()), createdAt DateTime @default(now()), updatedAt DateTime @updatedAt }`.

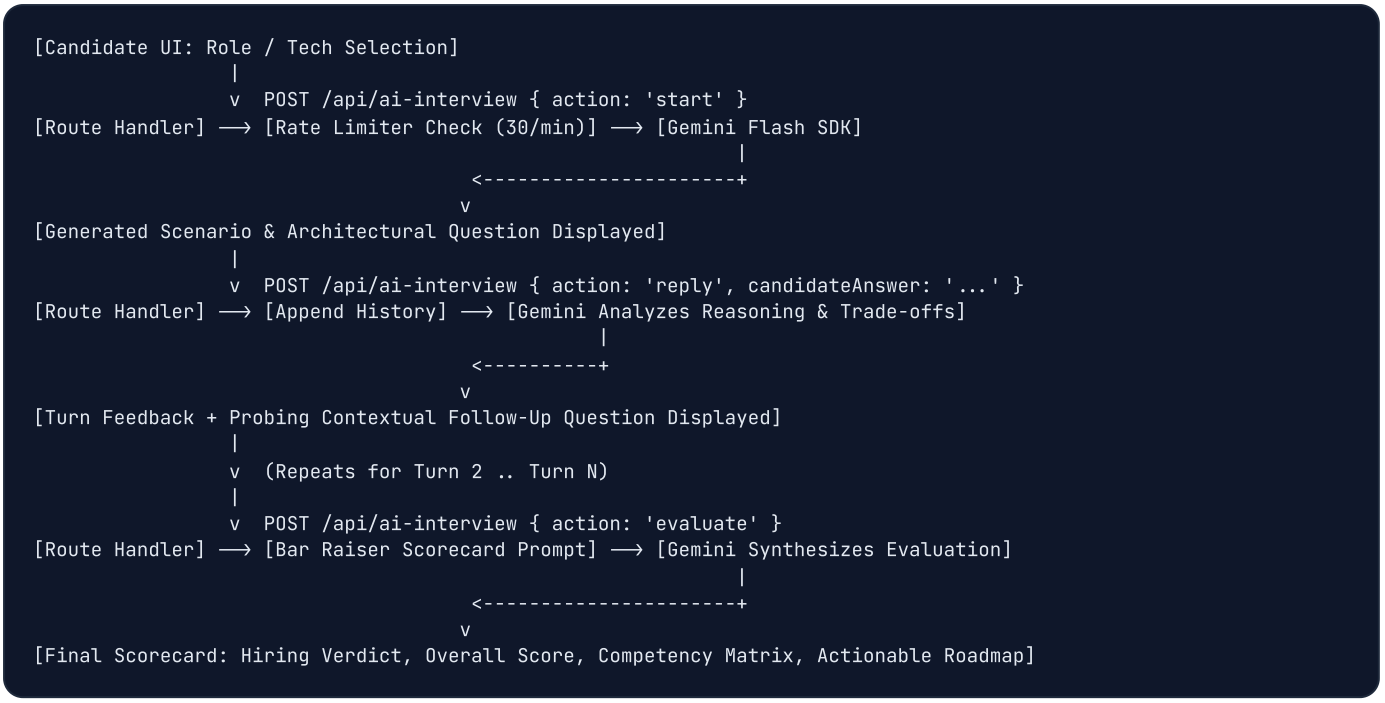
Security Rationale: Why Arbitrary Code Never Executes on the Server

Security Boundary Isolation: In many naive interview platforms, user-submitted code is sent to the backend and executed inside containers. This introduces container breakout vulnerabilities, crypto-mining attacks, and severe resource exhaustion. In Full Stack Interview Lab, all user code executes strictly inside client-side isolated sandboxes (Web Workers and isolated IFrames) and in-browser SQL relational engines (AlaSQL). The server route handlers remain stateless, fast, and protected from arbitrary code execution.

2. Deep Dive: Conversational AI Interviewer

AI ARCHITECTURE

The **AI Interviewer** is powered by the official `@google/genai` JavaScript SDK and connected to Google's `gemini-3.5-flash` (with fallback to `gemini-2.0-flash`). It simulates an adaptive, multi-turn interview with a Principal Engineer or Hiring Committee Bar Raiser.



Key Engineering Pillars of the AI Interview Engine

1. Strict Server-Side Isolation of Credentials

The Gemini API key is read exclusively from `process.env.GEMINI_API_KEY` on the server side in `src/app/api/ai-interview/route.ts` and `src/app/api/sql/evaluate/route.ts`. The key is never serialized, never sent to client-side components, and never returned in network responses.

2. Multi-Turn Session State Machine

The interview state is tracked through a structured session state machine:

- `config`: Role (e.g. Senior Full-Stack Engineer L5, Database Architect L6), Technology Domain, Seniority Bar, and Turn Limit.
- `currentQuestion`: The dynamic scenario statement generated by Gemini.
- `history`: The chronologically preserved conversation turns (Candidate answers and AI questions).
- `turnFeedback`: Per-turn technical critique evaluating trade-offs, potential edge cases, and architectural reasoning.
- `finalAssessment`: Generated upon interview conclusion across 4 key dimensions: System Architecture, Correctness & Edge Cases, Production Readiness, and Trade-off Analysis.

3. High-Fidelity Deterministic Offline Fallback

> **Zero-Downtime Guarantee:** If `GEMINI_API_KEY` is not provided, if the API rate limit is reached, or if the client is working in an offline environment, the system gracefully shifts to the deterministic offline simulator in

> `src/lib/gemini/offlineFallback.ts` and `src/lib/gemini/sqlPrompts.ts`. Candidates can continue practicing without UI crashes or frozen screens.

4. Rate Limiting & Timeout Protections

The service implements an in-memory sliding-window rate limiter restricting traffic to 30 requests per minute per server process. Every outbound call to Google Gemini is wrapped with a strict 35-second timeout guard using `AbortSignal.timeout(35000)` to prevent hanging connections.

3. SQL & PostgreSQL Interview Track (120 Questions)

DATABASE TRACK

The **SQL and PostgreSQL Interview Track** is a comprehensive database engineering curriculum designed to take candidates from relational fundamentals to staff-level production database engineering. It comprises **120 original questions** evenly distributed across 4 seniority tiers and 18 specialized architectural domains.

30	30	30	30	33	10
TIER 1: BEGINNER SQL	TIER 2: INTERMEDIATE SQL	TIER 3: ADVANCED POSTGRES	TIER 4: SENIOR SRE / DBA	EXECUTABLE SANDBOXES	INCIDENT COCKPITS

The 4 Progression Tiers

- **Tier 1: Beginner SQL (30 Qs):** Relational fundamentals, SELECT, WHERE, ORDER BY, LIMIT, INNER/LEFT/RIGHT JOINs, GROUP BY & HAVING, NULL semantics (IS NULL, COALESCE), and Primary/Foreign Key constraints.
- **Tier 2: Intermediate SQL (30 Qs):** Analytical window functions (DENSE_RANK, SUM OVER, LAG/LEAD), subqueries, Common Table Expressions (CTEs), ACID transaction boundaries, surrogate vs natural keys, UUIDv4 page fragmentation, and ON CONFLICT upsert mechanics.
- **Tier 3: Advanced PostgreSQL Internals (30 Qs):** B-Tree leaf pages, page splits, fillfactor tuning, EXPLAIN (ANALYZE, BUFFERS) query plan interpretation, GIN/GiST/BRIN index architectures, MVCC tuple headers (xmin, xmax, t_ctid), declarative table partitioning, PgBouncer pool modes, and Row-Level Security (RLS).
- **Tier 4: Senior PostgreSQL & Production Engineering (30 Qs):** Multi-transaction deadlock graph post-mortems, zero-downtime online schema migrations (expand/contract column rename), replication lag mitigation, table bloat remediation (pg_repack vs VACUUM FULL), checkpoint WAL bursts, disaster Point-In-Time Recovery (PITR), and high-availability Patroni split-brain fencing.

3 Real-World Relational Practice Datasets

The track embeds 3 rich relational datasets with comprehensive DDL schemas, primary keys, foreign keys, CHECK constraints, and realistic seed data:

DATASET DOMAIN	TABLES & RELATIONAL ENTITIES	TESTING SCENARIOS & REALISTIC BUSINESS LOGIC
E-COMMERCE	`customers`, `products`, `orders`, `order_items`, `payments`, `inventory`, `refunds` (7 tables)	Multi-item basket calculations, customer lifetime value (LTV), inventory stock reservation, partial refunds, payment gateway status reconciliation, and duplicate order detection.
LOAN MANAGEMENT NBFC	`customers`, `loan_applications`, `loans`, `emi_schedule`, `repayments`, `transactions` (6 tables)	Underwriting credit workflows, monthly EMI amortisation schedules, payment allocation (fees > interest > principal), 30/60/90 DPD delinquency tracking, and overdraft penalties.
SAAS ANALYTICS	`users`, `subscriptions`, `events`, `invoices`, `audit_logs` (5 tables)	Monthly Recurring Revenue (MRR), subscription tier transitions (trial, active, past_due, canceled), user retention cohort analysis, high-throughput event telemetry, and compliance audit trails.

Dual Execution Sandbox & AI Evaluation Architecture

> **In-Browser AlaSQL Sandbox vs Real PostgreSQL:**
> The platform explicitly distinguishes generic SQL practice from engine-specific PostgreSQL practice. Executable SQL queries run client-side inside an in-browser AlaSQL-99 relational engine with schema introspection, query timeout limits, and row-order normalization. For PostgreSQL-specific architecture (e.g. WAL, MVCC, B-Tree splits, deadlock graphs), the candidate submits their query plan or diagnostic strategy to the **Gemini AI Database Evaluator**

(`/api/sql/evaluate`), which analyzes query optimality, index usage, and production failure risks against an authoritative rubric.

Complete SQL & PostgreSQL Track Catalog (120 Questions)

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
TIER 1	SQL FUNDAMENTALS	sql-t1-01	Basic Column Projection and Row Limiting	INTERACTIVE SQL
TIER 1	SQL FUNDAMENTALS	sql-t1-02	Filtering Active Catalog Products with WHERE	INTERACTIVE SQL
TIER 1	SQL FUNDAMENTALS	sql-t1-03	Eliminating Duplicates with DISTINCT	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-04	Pattern Matching with LIKE and Wildcards	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-05	Range Filtering with BETWEEN and Date Comparison	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-06	Membership Filtering with the IN Operator	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-07	Handling Missing Data with IS NULL and IS NOT NULL	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-08	Aggregating Totals with COUNT, SUM, and AVG	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-09	Grouping and Aggregating with GROUP BY	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-10	Filtering Grouped Aggregates with HAVING	INTERACTIVE SQL
TIER 1	JOINS AND RELATIONSHIPS	sql-t1-11	Basic INNER JOIN Across Two Relational Tables	INTERACTIVE SQL
TIER 1	JOINS AND RELATIONSHIPS	sql-t1-12	Finding Inactive Records with LEFT JOIN and IS NULL	INTERACTIVE SQL
TIER 1	JOINS AND RELATIONSHIPS	sql-t1-13	Multi-Table Join: Order Items with Products and Customers	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-14	Loan Approval Rates and Risk Categorization	INTERACTIVE SQL
TIER 1	FILTERING, SORTING, AND AGGREGATION	sql-t1-15	Conditional Aggregation with CASE WHEN	INTERACTIVE SQL
TIER 1	DATABASE DESIGN AND NORMALIZATION	sql-t1-16	Relational Database Keys: Primary Keys vs. Foreign Keys	CONCEPTUAL
TIER 1	DATABASE DESIGN AND NORMALIZATION	sql-t1-17	First Normal Form (1NF) Fundamentals	CONCEPTUAL
TIER 1	DATABASE DESIGN AND NORMALIZATION	sql-t1-18	Second Normal Form (2NF) and Composite Keys	CONCEPTUAL
TIER 1	DATABASE DESIGN AND NORMALIZATION	sql-t1-19	Third Normal Form (3NF) and Transitive Dependencies	CONCEPTUAL

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
TIER 1	POSTGRESQL DATA TYPES	sql-t1-20	SQL Data Types: VARCHAR, CHAR, and TEXT in PostgreSQL	CONCEPTUAL
TIER 1	POSTGRESQL DATA TYPES	sql-t1-21	Date and Time Types: TIMESTAMP vs. TIMESTAMPTZ	CONCEPTUAL
TIER 1	TRANSACTIONS AND ACID	sql-t1-22	ACID Properties in Relational Databases	CONCEPTUAL
TIER 1	TRANSACTIONS AND ACID	sql-t1-23	Transactions with BEGIN, COMMIT, and ROLLBACK	CONCEPTUAL
TIER 1	INDEXES AND QUERY PLANNING	sql-t1-24	B-Tree Indexing Fundamentals	CONCEPTUAL
TIER 1	DATABASE DESIGN AND NORMALIZATION	sql-t1-25	Database Constraints: NOT NULL, UNIQUE, and CHECK	CONCEPTUAL
TIER 1	DATABASE SECURITY AND PERMISSIONS	sql-t1-26	SQL Injection Prevention: Parameterized Queries vs String Concatenation	CONCEPTUAL
TIER 1	SQL FUNDAMENTALS	sql-t1-27	Understanding Database Views	CONCEPTUAL
TIER 1	SQL FUNDAMENTALS	sql-t1-28	PostgreSQL Boolean Logic: COALESCE and NULLIF	CONCEPTUAL
TIER 1	POSTGRESQL DATA TYPES	sql-t1-29	Row Insertion and Returning Generated IDs with RETURNING	CONCEPTUAL
TIER 1	INDEXES AND QUERY PLANNING	sql-t1-30	Composite and Foreign Key Indexing Basics	CONCEPTUAL
TIER 2	SUBQUERIES AND CTES	sql-t2-01	Filtering Above-Average Values with Scalar Subqueries	INTERACTIVE SQL
TIER 2	SUBQUERIES AND CTES	sql-t2-02	Correlated Subquery with EXISTS	INTERACTIVE SQL
TIER 2	SUBQUERIES AND CTES	sql-t2-03	Common Table Expressions (CTEs) for Multi-Step Aggregations	INTERACTIVE SQL
TIER 2	WINDOW FUNCTIONS	sql-t2-04	Ranking Products Within Categories Using DENSE_RANK()	INTERACTIVE SQL
TIER 2	WINDOW FUNCTIONS	sql-t2-05	Calculating Running Totals with Cumulative Window Aggregation	INTERACTIVE SQL
TIER 2	WINDOW FUNCTIONS	sql-t2-06	Prior Period Comparison with LAG()	INTERACTIVE SQL
TIER 2	WINDOW FUNCTIONS	sql-t2-07	Top N Records Per Group Pattern	INTERACTIVE SQL
TIER 2	JOINS AND RELATIONSHIPS	sql-t2-08	FULL OUTER JOIN for Data Reconciliation	INTERACTIVE SQL
TIER 2	SQL FUNDAMENTALS	sql-t2-09	Combining Result Sets with UNION and UNION ALL	INTERACTIVE SQL

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
TIER 2	JOINS AND RELATIONSHIPS	sql-t2-10	Self-Joins for Hierarchical and Comparative Analysis	CONCEPTUAL
TIER 2	DATABASE DESIGN AND NORMALIZATION	sql-t2-11	Surrogate Keys vs. Natural Keys	CONCEPTUAL
TIER 2	POSTGRESQL DATA TYPES	sql-t2-12	UUIDv4 vs. BIGINT / ULID for Primary Keys	CONCEPTUAL
TIER 2	DATABASE DESIGN AND NORMALIZATION	sql-t2-13	Foreign Key Cascade Behaviors: CASCADE, SET NULL, and RESTRICT	CONCEPTUAL
TIER 2	TRANSACTIONS AND ACID	sql-t2-14	The Transactional Outbox Pattern in Relational Databases	CONCEPTUAL
TIER 2	DATABASE DESIGN AND NORMALIZATION	sql-t2-15	Database Normalization vs. Denormalization Trade-offs	CONCEPTUAL
TIER 2	INDEXES AND QUERY PLANNING	sql-t2-16	Partial Indexes in PostgreSQL	CONCEPTUAL
TIER 2	INDEXES AND QUERY PLANNING	sql-t2-17	Expression / Functional Indexes in PostgreSQL	CONCEPTUAL
TIER 2	JSONB AND POSTGRESQL-SPECIFIC FEATURES	sql-t2-18	PostgreSQL JSONB vs JSON Storage and Operators	CONCEPTUAL
TIER 2	JSONB AND POSTGRESQL-SPECIFIC FEATURES	sql-t2-19	PostgreSQL GIN (Generalized Inverted) Indexes on JSONB	CONCEPTUAL
TIER 2	ISOLATION LEVELS AND MVCC	sql-t2-20	PostgreSQL Read Committed vs. Repeatable Read Isolation Levels	CONCEPTUAL
TIER 2	LOCKS AND DEADLOCKS	sql-t2-21	Row-Level Locking with SELECT FOR UPDATE	CONCEPTUAL
TIER 2	TRANSACTIONS AND ACID	sql-t2-22	Optimistic Locking with Version Columns	CONCEPTUAL
TIER 2	ISOLATION LEVELS AND MVCC	sql-t2-23	Understanding PostgreSQL MVCC: Tuple Headers and xmin / xmax	CONCEPTUAL
TIER 2	ISOLATION LEVELS AND MVCC	sql-t2-24	PostgreSQL Autovacuum and Table Bloat	CONCEPTUAL
TIER 2	QUERY OPTIMIZATION	sql-t2-25	Keyset Pagination vs. OFFSET Pagination	CONCEPTUAL
TIER 2	LOCKS AND DEADLOCKS	sql-t2-26	Database Deadlocks: Causes and Prevention	CONCEPTUAL
TIER 2	BACKUP, RECOVERY, AND REPLICATION	sql-t2-27	PostgreSQL Read Replicas and Replication Lag	CONCEPTUAL
TIER 2	SQL FUNDAMENTALS	sql-t2-28	SQL Upsert Patterns: INSERT ... ON CONFLICT DO UPDATE	CONCEPTUAL
TIER 2	DATABASE DESIGN AND NORMALIZATION	sql-t2-29	Soft Deletes vs. Hard Deletes: Architecture and Query Traps	CONCEPTUAL

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
TIER 2	QUERY OPTIMIZATION	sql-t2-30	Materialized Views and Concurrent Refresh in PostgreSQL	CONCEPTUAL
TIER 3	INDEXES AND QUERY PLANNING	sql-tier3-001	B-Tree Index Internals: Leaf Pages, Splits, and Fillfactor	CONCEPTUAL
TIER 3	INDEXES AND QUERY PLANNING	sql-tier3-002	EXPLAIN (ANALYZE, BUFFERS): Plan Reading & Scan Types	CONCEPTUAL
TIER 3	INDEXES AND QUERY PLANNING	sql-tier3-003	GIN vs GiST vs BRIN Indexes: Architecture and Selection	CONCEPTUAL
TIER 3	ISOLATION LEVELS AND MVCC	sql-tier3-004	PostgreSQL MVCC Internals: Heap Tuples, xmin, xmax, and Hints	CONCEPTUAL
TIER 3	PRODUCTION DEBUGGING	sql-tier3-005	Autovacuum Tuning, Table Bloat, and Freeze Wraparound	CONCEPTUAL
TIER 3	PARTITIONING AND LARGE TABLES	sql-tier3-006	Declarative Partitioning: Range, List, and Hash Strategies	CONCEPTUAL
TIER 3	JSONB AND POSTGRESQL-SPECIFIC FEATURES	sql-tier3-007	PostgreSQL JSONB: Operators, jsonb_path_query, and GIN Indexing	CONCEPTUAL
TIER 3	ISOLATION LEVELS AND MVCC	sql-tier3-008	PostgreSQL Transaction Isolation: Write Skew and Serializable Snapshot Isolation (SSI)	CONCEPTUAL
TIER 3	LOCKS AND DEADLOCKS	sql-tier3-009	Advisory Locks: Application-Level Distributed Locking in Postgres	CONCEPTUAL
TIER 3	SUBQUERIES AND CTES	sql-tier3-010	Hierarchical Queries with Recursive CTEs (WITH RECURSIVE)	INTERACTIVE SQL
TIER 3	DATABASE SECURITY AND PERMISSIONS	sql-tier3-011	Row-Level Security (RLS) for Multi-Tenant Database Architecture	CONCEPTUAL
TIER 3	QUERY OPTIMIZATION	sql-tier3-012	PgBouncer Connection Pooling: Session vs Transaction vs Statement Modes	CONCEPTUAL
TIER 3	POSTGRESQL DATA TYPES	sql-tier3-013	PostgreSQL LISTEN and NOTIFY: Pub/Sub Architecture and Queue Hazards	CONCEPTUAL
TIER 3	QUERY OPTIMIZATION	sql-tier3-014	Materialized Views with REFRESH CONCURRENTLY	CONCEPTUAL
TIER 3	WINDOW FUNCTIONS	sql-tier3-015	Window Frame Specifications: ROWS vs RANGE vs GROUPS	INTERACTIVE SQL
TIER 3	QUERY OPTIMIZATION	sql-tier3-016	PostgreSQL Join Strategies: Nested Loop, Hash Join, and	CONCEPTUAL

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
			Merge Join	
TIER 3	MIGRATIONS AND SCHEMA EVOLUTION	sql-tier3-017	Zero-Downtime Schema Migrations: Safe Index Creation & Column Additions	CONCEPTUAL
TIER 3	LOCKS AND DEADLOCKS	sql-tier3-018	Advisory Locks vs FOR UPDATE SKIP LOCKED for Message Queues	INTERACTIVE SQL
TIER 3	POSTGRESQL DATA TYPES	sql-tier3-019	PostgreSQL Full-Text Search: tsvector, tsquery, and GIN Indexing	CONCEPTUAL
TIER 3	SUBQUERIES AND CTES	sql-tier3-020	CTEs in PostgreSQL 12+: MATERIALIZED vs NOT MATERIALIZED	CONCEPTUAL
TIER 3	DATABASE DESIGN AND NORMALIZATION	sql-tier3-021	PostgreSQL Foreign Data Wrappers (postgres_fdw) and Pushdown	CONCEPTUAL
TIER 3	MIGRATIONS AND SCHEMA EVOLUTION	sql-tier3-022	Audit Logging with Trigger Functions and pg_trigger_depth	CONCEPTUAL
TIER 3	PRODUCTION DEBUGGING	sql-tier3-023	PostgreSQL Extensions: pg_stat_statements and Performance Profiling	CONCEPTUAL
TIER 3	POSTGRESQL DATA TYPES	sql-tier3-024	Custom PostgreSQL Types: DOMAIN vs ENUM vs Composite Types	CONCEPTUAL
TIER 3	INDEXES AND QUERY PLANNING	sql-tier3-025	Partial Indexes and Expression Indexes: Advanced Query Optimization	INTERACTIVE SQL
TIER 3	QUERY OPTIMIZATION	sql-tier3-026	PostgreSQL JIT Compilation: Overhead, Thresholds, and OLTP Regressions	CONCEPTUAL
TIER 3	POSTGRESQL DATA TYPES	sql-tier3-027	Array Data Types and High-Performance Containment Queries	INTERACTIVE SQL
TIER 3	INDEXES AND QUERY PLANNING	sql-tier3-028	PostgreSQL NULLS FIRST / NULLS LAST Index Semantics	CONCEPTUAL
TIER 3	DATABASE SECURITY AND PERMISSIONS	sql-tier3-029	Database Security: Role Inheritance, Schema search_path Poisoning, and SECURITY DEFINER	CONCEPTUAL
TIER 3	LOCKS AND DEADLOCKS	sql-tier3-030	Advisory Locks for Distributed Idempotency and Rate Limiting	CONCEPTUAL
TIER 4	LOCKS AND DEADLOCKS	sql-tier4-001	Production Incident: Multi-Transaction Deadlock Graph & Lock Queue Analysis	SRE INCIDENT
TIER 4	MIGRATIONS AND SCHEMA EVOLUTION	sql-tier4-002	Zero-Downtime Column Rename and Type Migration in 24/7 Production	CONCEPTUAL

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
TIER 4	BACKUP, RECOVERY, AND REPLICATION	sql-tier4-003	Production Incident: Replication Lag Spike & Disk Exhaustion on Primary	SRE INCIDENT
TIER 4	PRODUCTION DEBUGGING	sql-tier4-004	Zero-Downtime Table Bloat Remediation: pg_repack vs VACUUM FULL	CONCEPTUAL
TIER 4	QUERY OPTIMIZATION	sql-tier4-005	Checkpoint Tuning & WAL Generation Spikes: Eliminating I/O Spikes	CONCEPTUAL
TIER 4	BACKUP, RECOVERY, AND REPLICATION	sql-tier4-006	Point-In-Time Recovery (PITR) Disaster Recovery Architecture	CONCEPTUAL
TIER 4	PRODUCTION DEBUGGING	sql-tier4-007	Production Incident: Stalled Autovacuum & Impending 2-Billion XID Wraparound	SRE INCIDENT
TIER 4	INDEXES AND QUERY PLANNING	sql-tier4-008	Corrupt Index Remediation and REINDEX CONCURRENTLY Failures	SRE INCIDENT
TIER 4	QUERY OPTIMIZATION	sql-tier4-009	Atomic Distributed Rate Limiting & Token Bucket in Pure PostgreSQL	INTERACTIVE SQL
TIER 4	DATABASE DESIGN AND NORMALIZATION	sql-tier4-010	Multi-Tenant Database Architectures: Trade-Off Analysis at Scale	CONCEPTUAL
TIER 4	LOCKS AND DEADLOCKS	sql-tier4-011	Production Incident: Cascading Lock Pileup Caused by Missing Foreign Key Index	SRE INCIDENT
TIER 4	TRANSACTIONS AND ACID	sql-tier4-012	Two-Phase Commit (2PC) and Orphaned Prepared Transaction Disasters	SRE INCIDENT
TIER 4	PRODUCTION DEBUGGING	sql-tier4-013	Linux OOM Killer Tuning, overcommit_memory, and PostgreSQL Work_mem	SRE INCIDENT
TIER 4	QUERY OPTIMIZATION	sql-tier4-014	Query Plan Regressions: Parameter Sniffing & Generic vs Custom Plans	CONCEPTUAL
TIER 4	PARTITIONING AND LARGE TABLES	sql-tier4-015	High-Throughput TimescaleDB / Hypertable Architecture vs Native Partitioning	CONCEPTUAL
TIER 4	QUERY OPTIMIZATION	sql-tier4-016	Zero-Downtime Deduplication of 50 Million Records Under Live Write Load	INTERACTIVE SQL
TIER 4	QUERY OPTIMIZATION	sql-tier4-017	Correlated Columns and Multi-Column Extended Statistics (CREATE STATISTICS)	CONCEPTUAL

TIER	CATEGORY	IDENTIFIER	PROBLEM / SCENARIO TITLE	TYPE
TIER 4	BACKUP, RECOVERY, AND REPLICATION	sql-tier4-018	PostgreSQL Logical Decoding and Change Data Capture (CDC) Architecture	CONCEPTUAL
TIER 4	POSTGRESQL DATA TYPES	sql-tier4-019	High-Performance Unlogged Tables vs Temp Tables for Ephemeral Staging	CONCEPTUAL
TIER 4	DATABASE SECURITY AND PERMISSIONS	sql-tier4-020	Database Security Hardening: SCRAM-SHA-256, Connection Quotas, and pgaudit	CONCEPTUAL
TIER 4	LOCKS AND DEADLOCKS	sql-tier4-021	Production Incident: Statement Timeout vs Lock Timeout Cascading Outage	SRE INCIDENT
TIER 4	BACKUP, RECOVERY, AND REPLICATION	sql-tier4-022	High-Availability Failover Architectures: Patroni, Raft Quorum, and Split-Brain Defense	CONCEPTUAL
TIER 4	QUERY OPTIMIZATION	sql-tier4-023	Heap-Only Tuple (HOT) Optimization Failures and Write Amplification	CONCEPTUAL
TIER 4	QUERY OPTIMIZATION	sql-tier4-024	PostgreSQL Memory Hierarchy: Shared Buffers, OS Page Cache, and Double Buffering	CONCEPTUAL
TIER 4	LOCKS AND DEADLOCKS	sql-tier4-025	PostgreSQL Advisory Locks for Queue Worker Deadlock Prevention	INTERACTIVE SQL
TIER 4	BACKUP, RECOVERY, AND REPLICATION	sql-tier4-026	Production Incident: Replica WAL Replay Lag Induced by Long Standby Queries	SRE INCIDENT
TIER 4	QUERY OPTIMIZATION	sql-tier4-027	Keyset Pagination vs Deep OFFSET: Eliminating O(N) Query Degradation	INTERACTIVE SQL
TIER 4	DATABASE DESIGN AND NORMALIZATION	sql-tier4-028	PostgreSQL Foreign Keys with ON DELETE CASCADE: Lock Cascade Hazards	CONCEPTUAL
TIER 4	MIGRATIONS AND SCHEMA EVOLUTION	sql-tier4-029	Zero-Downtime NOT NULL Constraint Addition on 100-Million Row Tables	CONCEPTUAL
TIER 4	BACKUP, RECOVERY, AND REPLICATION	sql-tier4-030	PostgreSQL Disaster Post-Mortem: Dropped Table Recovery Without Backup Restore	SRE INCIDENT

4. Senior & Staff Challenge Library (51 Scenarios)

51 CHALLENGES

The library contains **51 production-grade engineering scenarios** covering deep runtime behaviors, deadlock mitigation, distributed systems, and real-world incidents.

DOMAIN	IDENTIFIER	SCENARIO TITLE	DIFFICULTY	FORMAT
JAVASCRIPT	js-event-loop-microtasks	Flash-Sale Batch Processor Microtask Queue Starvation	SENIOR	DEBUGGING
JAVASCRIPT	js-closures-memory-leak	SSE Connection Manager Request Context Closure Leak	SENIOR	DEBUGGING
JAVASCRIPT	js-promise-race-condition	Multi-Tab Wallet Checkout Balance Debit Race Condition	SENIOR	CODING
JAVASCRIPT	js-async-error-propagation	Background Reconciliation Pipeline Unhandled Rejection Crash	SENIOR	DEBUGGING
JAVASCRIPT	js-perf-object-allocation	High-Throughput Metrics Aggregator Garbage Collection Latency Spikes	SENIOR	DEBUGGING
TYPESCRIPT	ts-discriminated-unions-events	Multi-Gateway Webhook Event Normalization & Exhaustive Narrowing	SENIOR	CODING
TYPESCRIPT	ts-generics-api-client	Type-Safe RPC/REST Client with Path Parameter Inference	SENIOR	CODING
TYPESCRIPT	ts-type-narrowing-rbac	Enterprise Multi-Tenant RBAC Authorization Type Narrowing	SENIOR	CODING
TYPESCRIPT	ts-deep-immutability-state	Deep Readonly State Container with Lens-Based Immutability	SENIOR	CODING
REACT	react-stale-closure-interval	Order-Book Live Ticker Stale Closure Hook Bug	SENIOR	DEBUGGING
REACT	react-rerender-cascade-table	10,000-Row Logistics Inventory Table Cascading Re-Render Freeze	SENIOR	DEBUGGING
REACT	react-race-condition-autocomplete	Customer Support CRM Search Out-of-Order Response Overwrite	SENIOR	DEBUGGING
REACT	react-accessible-combobox	Accessible ARIA 1.2 Enterprise Multi-Select Combobox	SENIOR	CODING
REACT	react-custom-hook-optimistic-mutation	E-Commerce Cart Optimistic Mutation Hook with Rollback	SENIOR	CODING
REACT	enterprise-loan-management	Enterprise Loan Management Dashboard (React & Redux Toolkit)	SENIOR	CODING

DOMAIN	IDENTIFIER	SCENARIO TITLE	DIFFICULTY	FORMAT
NEXT.JS	nextjs-server-client-boundary	Secret Key Leakage Across Next.js Server-Client Boundary	SENIOR	DEBUGGING
NEXT.JS	nextjs-hydration-mismatch-clock	Multi-Timezone Scheduling Hydration Mismatch & Layout Shift	SENIOR	DEBUGGING
NEXT.JS	nextjs-isr-cache-stampede	Product Catalog High-Traffic ISR Cache Stampede / Dog-Piling	SENIOR	DEBUGGING
NEXT.JS	nextjs-auth-middleware-session	Edge Middleware JWT Validation Infinite Redirect Loop	SENIOR	DEBUGGING
NEXT.JS	nextjs-stream-suspense-error-boundary	AI Streaming Assistant Suspense & Localized Error Boundary Fallback	SENIOR	CODING
NODE.JS	nodejs-event-loop-blocking-crypto	Auth Gateway Event-Loop Freezing on CPU-Intensive Hashing	SENIOR	DEBUGGING
NODE.JS	nodejs-express-error-middleware	Express Asynchronous Error Swallowing & Hanging Socket Leak	SENIOR	DEBUGGING
NODE.JS	nodejs-rate-limiting-redis	Distributed Sliding Window Rate Limiter with Redis Race Conditions	SENIOR	CODING
NODE.JS	nodejs-stream-backpressure	Multi-Gigabyte Financial Audit CSV Export Out-of-Memory (OOM) Crash	SENIOR	DEBUGGING
NODE.JS	nodejs-graceful-shutdown-worker	Kubernetes Rolling Deployment Background Transaction Data Loss	SENIOR	DEBUGGING
DATABASES	db-deadlock-inventory-reservation	Flash Sale Cart Reservation PostgreSQL Transaction Deadlock	SENIOR	DEBUGGING
DATABASES	db-n-plus-one-query-plan	E-Commerce Order History Endpoint N+1 Query & Composite Index Optimization	SENIOR	DEBUGGING
DATABASES	db-isolation-levels-phantom-read	Financial Ledger Wallet Balance Anomaly & Isolation Levels	SENIOR	DEBUGGING
DATABASES	db-keyset-pagination-performance	CRM Million-Row Activity Log OFFSET Pagination Degradation	SENIOR	DEBUGGING
DATABASES	db-zero-downtime-migration	50M-Row User Schema Column Rename Zero-Downtime Migration	SENIOR	SYSTEM_DESIGN
INTEGRATIONS	integ-webhook-idempotency-replay	Stripe Payment Webhook Replay & Duplicate Credit Crediting	SENIOR	DEBUGGING

DOMAIN	IDENTIFIER	SCENARIO TITLE	DIFFICULTY	FORMAT
INTEGRATIONS	integ-whatsapp-sms-fallback	Resilient Multi-Provider WhatsApp & SMS Fallback Orchestrator	SENIOR	CODING
INTEGRATIONS	integ-crm-sync-out-of-order	Bi-Directional CRM Sync Out-of-Order Customer Data Rollback	SENIOR	DEBUGGING
INTEGRATIONS	integ-exponential-backoff-jitter	Third-Party Shipping API Retry Storm & Thundering Herd Incident	SENIOR	DEBUGGING
ENGINEERING OPERATIONS	ops-production-memory-leak-debug	Production Incident: Node.js Container OOMKilled Memory Leak	SENIOR	INCIDENT
ENGINEERING OPERATIONS	ops-secrets-leak-incident	Production Incident: Cloud Provider IAM & Stripe Key Leak in CI Logs	SENIOR	INCIDENT
ENGINEERING OPERATIONS	ops-canary-rollback-metrics	Canary Deployment 5xx Spike & Automated Rollback Analysis	SENIOR	DEBUGGING
ENGINEERING OPERATIONS	ops-distributed-tracing-bottleneck	OpenTelemetry Trace Bottleneck Analysis in Microservice Mesh	SENIOR	DEBUGGING
SYSTEM DESIGN	sysdesign-scalable-notification-engine	Global Multi-Channel Notification Platform (50M Events/Day)	STAFF/PRINCIPAL	SYSTEM_DESIGN
SYSTEM DESIGN	sysdesign-idempotent-checkout-service	High-Volume Idempotent Checkout & Order Processing System	STAFF/PRINCIPAL	SYSTEM_DESIGN
SYSTEM DESIGN	sysdesign-multi-tenant-crm-analytics	Real-Time Multi-Tenant CRM Analytics & Aggregation Engine	STAFF/PRINCIPAL	SYSTEM_DESIGN
SYSTEM DESIGN	sysdesign-distributed-job-scheduler	Distributed Cron & Delayed Job Scheduler (At-Least-Once Delivery)	SENIOR	SYSTEM_DESIGN
SECURITY	review-pr-payment-split	Code Review PR #412: Marketplace Payout Split & Transfer Engine	SENIOR	CODE_REVIEW
SECURITY	review-pr-session-revocation	Code Review PR #589: Distributed Redis Session Invalidation	SENIOR	CODE_REVIEW
SYSTEM DESIGN	interview-staff-architect-modernization	Staff Architect Interview: Legacy Monolith Modernization to Event-Driven Architecture	STAFF/PRINCIPAL	SENIOR_INTERVIEW
ENGINEERING OPERATIONS	interview-lead-incident-postmortem	Lead Engineer Interview: 45-Minute Outage Facilitation & Blameless Post-Mortem	STAFF/PRINCIPAL	SENIOR_INTERVIEW
SECURITY	sec-ssrf-webhook-metadata	Custom Webhook Dispatcher AWS Metadata SSRF Vulnerability	SENIOR	DEBUGGING

DOMAIN	IDENTIFIER	SCENARIO TITLE	DIFFICULTY	FORMAT
SYSTEM DESIGN	sysdesign-distributed-search-engine	**E-Commerce Real-Time Full-Text Search & Faceted Filtering**	STAFF/PRINCIPAL	SYSTEM_DESIGN
INTEGRATIONS	integ-payment-reconciliation-ledger	**Automated Nightly Payment Gateway Reconciliation & Variance Ledger**	SENIOR	CODING
ENGINEERING OPERATIONS	ops-secrets-rotation-vault	**Zero-Downtime Database Secret Rotation with HashiCorp Vault**	SENIOR	DEBUGGING
NODE.JS	node-rest-api-validation-zod	**High-Throughput REST Ingestion & Zod Schema Validation Guard**	SENIOR	CODING

5. Tier 1 Beginner Track Catalog (63 Questions)

63 QUESTIONS

The **Tier 1 Beginner Track** is built specifically for junior and mid-level full-stack engineers to solidify their fundamental understanding. It spans 6 critical categories with interactive explanations, progressive guidance hints, and starter coding tasks.

CATEGORY	QUESTION ID	QUESTION TITLE	CODE TASK
JAVASCRIPT FUNDAMENTALS	t1-js-01	What is the difference between var, let, and const in JavaScript?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-02	What are JavaScript data types and how does primitive vs reference comparison work?	CONCEPTUAL
JAVASCRIPT FUNDAMENTALS	t1-js-03	What is the difference between loose equality (==) and strict equality (===)?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-04	How do if/else statements and switch statements differ in JavaScript?	CONCEPTUAL
JAVASCRIPT FUNDAMENTALS	t1-js-05	What are the differences between for, while, and for...of loops?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-06	What is the difference between a regular function and an arrow function?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-07	How do map, filter, and find work on JavaScript arrays?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-08	How do you access, update, and destructure JavaScript object properties?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-09	What is variable scope and what is a closure in simple terms?	INTERACTIVE CODE
JAVASCRIPT FUNDAMENTALS	t1-js-10	What is hoisting in JavaScript?	CONCEPTUAL
JAVASCRIPT FUNDAMENTALS	t1-js-11	What are Promises and how does async/await make handling them easier?	CONCEPTUAL
JAVASCRIPT FUNDAMENTALS	t1-js-12	How does the JavaScript event loop work in simple terms?	CONCEPTUAL
HTML AND CSS	t1-hc-01	What is semantic HTML and why is it important?	CONCEPTUAL
HTML AND CSS	t1-hc-02	How do HTML forms work, and why should inputs always have associated labels?	INTERACTIVE CODE
HTML AND CSS	t1-hc-03	How does CSS specificity work and what are the main selector types?	CONCEPTUAL
HTML AND CSS	t1-hc-04	What is the CSS Box Model and what does box-sizing: border-box do?	CONCEPTUAL
HTML AND CSS	t1-hc-05	What is Flexbox and what are its core container and item properties?	INTERACTIVE CODE
HTML AND CSS	t1-hc-06	What is CSS Grid and when would you choose it over Flexbox?	CONCEPTUAL
HTML AND CSS	t1-hc-07	How do you create a responsive web design using media queries?	CONCEPTUAL

CATEGORY	QUESTION ID	QUESTION TITLE	CODE TASK
HTML AND CSS	t1-hc-08	What is the difference between px, rem, em, and % units in CSS?	CONCEPTUAL
HTML AND CSS	t1-hc-09	How do the different CSS position values work (static, relative, absolute, fixed, sticky)?	CONCEPTUAL
HTML AND CSS	t1-hc-10	What are the most reliable modern ways to center an element in CSS?	INTERACTIVE CODE
REACT FUNDAMENTALS	t1-react-01	What is React and why do engineers choose it over vanilla JavaScript for building UIs?	CONCEPTUAL
REACT FUNDAMENTALS	t1-react-02	What is JSX and what are its key syntax rules in React?	INTERACTIVE CODE
REACT FUNDAMENTALS	t1-react-03	What is the difference between props and state in React?	CONCEPTUAL
REACT FUNDAMENTALS	t1-react-04	How does event handling work in React, and why should you pass a function reference instead of invoking it?	INTERACTIVE CODE
REACT FUNDAMENTALS	t1-react-05	How do you conditionally render elements in React?	CONCEPTUAL
REACT FUNDAMENTALS	t1-react-06	How do you render lists in React, and why is the key prop strictly necessary?	INTERACTIVE CODE
REACT FUNDAMENTALS	t1-react-07	How does the useState hook work, and when should you use a functional state update?	INTERACTIVE CODE
REACT FUNDAMENTALS	t1-react-08	What is the useEffect hook, and how does its dependency array control execution?	CONCEPTUAL
REACT FUNDAMENTALS	t1-react-09	What is a controlled component in React and how does it handle form inputs?	INTERACTIVE CODE
REACT FUNDAMENTALS	t1-react-10	How does parent-to-child and child-to-parent communication work in React?	CONCEPTUAL
REACT FUNDAMENTALS	t1-react-11	What are the main component lifecycle concepts in modern functional React?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-01	Why is state management useful in web applications and what problem does it solve?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-02	What is the difference between local component state and global state?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-03	What are the three core principles and concepts of Redux: Store, Actions, and Reducers?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-04	What is createSlice in Redux Toolkit and how does it simplify Redux code?	INTERACTIVE CODE
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-05	How does Immer work inside Redux Toolkit reducers to allow "direct state mutation"?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-06	How do useSelector and useDispatch connect React components to the Redux store?	INTERACTIVE CODE
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-07	How do you handle asynchronous operations like API calls using createAsyncThunk?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-08	How do you configure the Redux store using configureStore and provide it to React?	CONCEPTUAL

CATEGORY	QUESTION ID	QUESTION TITLE	CODE TASK
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-09	When is Redux useful and when is it unnecessary / overkill?	CONCEPTUAL
REDUX TOOLKIT FUNDAMENTALS	t1-rdx-10	How do action payloads work in Redux Toolkit and how do you pass parameters to actions?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-01	What is TypeScript and what primary advantages does it offer over vanilla JavaScript?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-02	What are the basic primitive types in TypeScript and how do type annotations work?	INTERACTIVE CODE
TYPESCRIPT FUNDAMENTALS	t1-ts-03	What is the difference between an interface and a type alias in TypeScript?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-04	How do you type arrays and objects in TypeScript?	INTERACTIVE CODE
TYPESCRIPT FUNDAMENTALS	t1-ts-05	How do you type function parameters, return values, and functions that return nothing (void)?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-06	How do optional properties (?) and optional chaining (?.) work in TypeScript?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-07	What are union types and how do you narrow them using typeof guards?	INTERACTIVE CODE
TYPESCRIPT FUNDAMENTALS	t1-ts-08	What is the difference between any and unknown in TypeScript and why is unknown safer?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-09	What are literal types and how are they used to model exact state values?	CONCEPTUAL
TYPESCRIPT FUNDAMENTALS	t1-ts-10	What are basic Generics in TypeScript and why are they useful?	INTERACTIVE CODE
NODE.JS AND API FUNDAMENTALS	t1-node-01	What is Node.js and how does running JavaScript in Node differ from running it in a web browser?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-02	What is meant by the "non-blocking, event-driven I/O" model in Node.js?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-03	What is a REST API and what are its core architectural principles?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-04	What are the main HTTP methods and how do their purposes differ?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-05	What are the most common HTTP status code ranges and their specific meanings?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-06	How do you build a basic web server and define routes using Express.js?	INTERACTIVE CODE
NODE.JS AND API FUNDAMENTALS	t1-node-07	How do you extract data from req.params, req.query, and req.body in Express?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-08	What is Express middleware and what is the role of the next() function?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-09	What are environment variables, process.env, and why must secret keys never be committed to Git?	CONCEPTUAL
NODE.JS AND API FUNDAMENTALS	t1-node-10	What are the basic differences between Relational (SQL) and Document (NoSQL) databases for beginners?	CONCEPTUAL

6. Monaco Coding Practice & Enterprise Loan Lab

SPECIALIZED LABS

In-Browser Monaco Coding Practice Problems

Integrated Monaco coding environment with live in-browser execution sandboxes, sample inputs, expected outputs, hints, and automated test runners.

LANGUAGE	IDENTIFIER	PROBLEM TITLE	DIFFICULTY
JAVASCRIPT	`js-debounce-options`	**Advanced Debounce with Leading & Trailing Options**	SENIOR
JAVASCRIPT	`js-promise-concurrency-pool`	**Async Promise Concurrency Pool (Worker Throttler)**	SENIOR
TYPESCRIPT	`ts-deep-flatten-keys`	**Deep Nested Object Path Keys Union Type**	SENIOR
SQL	`sql-monthly-cohort-retention`	**Monthly User Cohort Retention & Churn Analysis**	SENIOR
SQL	`sql-top-n-per-category`	**Top 2 Revenue Products Per Category (Window Functions)**	SENIOR
PYTHON	`py-token-bucket-limiter`	**Python Token Bucket Distributed Rate Limiter**	SENIOR

Enterprise Loan Management Dashboard: 8 Production Concurrency Bug Scenarios

An enterprise Non-Banking Financial Company (NBFC) dashboard managing thousands of loan applications. The interactive **8 Production Bugs Explorer** challenges engineers to diagnose and resolve high-stakes frontend bugs:

BUG #	CATEGORY	PROBLEM TITLE	ROOT CAUSE & SENIOR MITIGATION STRATEGY
BUG #1	CONCURRENCY	**Stale Asynchronous Search Overwrites Newer Query Results** A user searches for "Aarav", then quickly deletes it and searches for "Sophia". The first slow API call finishes after the second fast call and overwrites the active table results with the stale query.	**Root Cause:** Asynchronous responses resolve in non-deterministic order (network jitter). Without an AbortController or a monotonic request-ID/sequence tracker in Redux state, the latest resolved promise always commits to state regardless of when it was initiated. **Senior Fix:** 1. Utilize Redux Toolkit `thunkAPI.signal` with `fetch(url, { signal: thunkAPI.signal })` or `AbortController` to abort in-flight requests when a new search query is dispatched. 2. In the Redux slice, track `latestRequestId: string | null` or monotonic `timestamp`. In the `extraReducers.fulfilled` matcher, compare `action.meta.requestId === state.latestRequestId`. If mismatched, silently ignore the stale payload.
BUG #2	PERFORMANCE	**Duplicate API Requests During Component Re-rendering & Navigation** Every time a parent component re-renders or the user tabs between drawer views, duplicate	**Root Cause:** `useEffect` dependency arrays include unmemoized object literals or functions (e.g. `[filters, pagination]`), triggering repeated execution on every

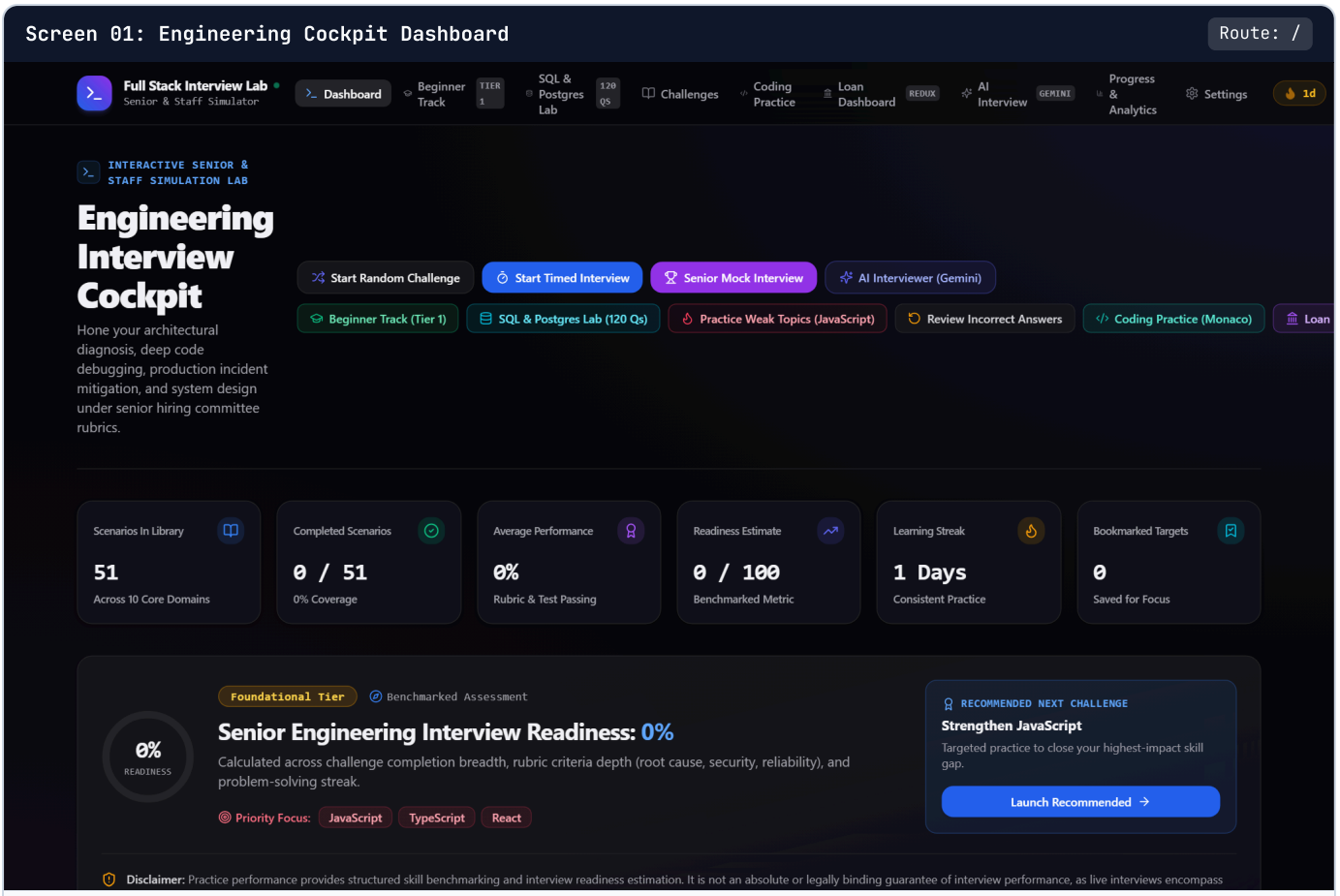
BUG #	CATEGORY	PROBLEM TITLE	ROOT CAUSE & SENIOR MITIGATION STRATEGY
		identical GET requests hit the `/api/applications` and `/api/dashboard/stats` endpoints.	render cycle even when actual property values have not changed. **Senior Fix:** 1. Normalize dependency primitives (e.g., `filters.search, filters.status, pagination.page, pagination.limit`) or use custom deep-equality comparisons. 2. In Redux thunks, leverage the `condition` option in `createAsyncThunk(type, payloadCreator, { condition: (arg, { getState }) ⇒ !isCurrentlyLoading })` to suppress duplicate in-flight dispatches.
BUG #3	STATE MANAGEMENT	**Optimistic Status Update Failure Without Safe Rollback** An officer approves a loan. The UI turns green immediately. The backend rejects the request due to an expired session or credit policy rule, but the UI remains "APPROVED", deceiving the officer.	**Root Cause:** Optimistic state mutation was applied on action dispatch, but no error rollback handler was implemented in `extraReducers.rejected` to restore the previous snapshot. **Senior Fix:** 1. When dispatching an optimistic update, capture a clone of the original application record in an `optimisticBackups: Record<string, LoanApplication>` map. 2. If the thunk resolves with `rejected`, look up the backup snapshot and restore the record into `state.items` atomically, dispatching a high-visibility error toast notification.
BUG #4	UX/PAGINATION	**Filter Change Leaves Current Page Out of Bounds** A user navigates to Page 6 of loan applications. They then apply a filter for `status === "REJECTED"`, which only has 2 matching records. The table displays "No records found" because it requested Page 6 of a 1-page result set.	**Root Cause:** The search and filter reducer actions modify `state.filters`, but fail to reset `state.pagination.page` to 1. **Senior Fix:** Any action that narrows or alters the query criteria (`setSearchQuery`, `setStatusFilter`, `setLoanTypeFilter`, `setPageSize`) must explicitly reset `state.pagination.page = 1`.
BUG #5	STATE MANAGEMENT	**Accidental Direct State Mutation Bypassing Immer or Corrupting State** A developer copies array references using shallow array methods (`const items = state.items; items.sort(...)`), mutating the frozen state in place or returning a new value while also mutating draft state in Redux Toolkit.	**Root Cause:** Array prototype methods like `.sort()`, `.reverse()`, and `.splice()` mutate arrays in place. In Redux Toolkit reducers, mixing `return newState` with direct mutations confuses Immer draft finalization. **Senior Fix:** Either mutate the Immer draft directly without returning, or return a complete new state object. When sorting or filtering arrays in reducers or selectors, always copy first (`[...state.items].sort(...)`).
BUG #6	PERFORMANCE	**Keystroke Search Causes Full Table and Metric Cards Re-render** Typing into the search input causes all 50 table rows, all 5 metric cards, and the drawer component to re-render on every single character, causing typing latency on low-power devices.	**Root Cause:** Parent component holds the un-debounced search input in parent state and passes raw inline object callbacks (`onClick={() ⇒ handleSelect(id)}`) to child rows that lack `React.memo`. Selectors return new object references on every run. **Senior Fix:** 1. Debounce the search input using a 300ms timer so Redux action is only dispatched once the user pauses. 2. Wrap `LoanTableRow` with `React.memo` and pass stable `useCallback` action handlers. 3. Use

BUG #	CATEGORY	PROBLEM TITLE	ROOT CAUSE & SENIOR MITIGATION STRATEGY
			`createSelector` from Redux Toolkit to memoize derived state calculations.
BUG #7	CONCURRENCY	**Asynchronous Callback Captures Outdated Filter or Pagination State** A background polling interval or debounced callback dispatches a refresh request using stale filter values captured when the callback was initially created.	**Root Cause:** A callback or `useEffect` timer enclosed `filters` without updating the closure, or did not read the latest state from `getState()` inside a Redux thunk. **Senior Fix:** Inside Redux thunks, always read `getState()` dynamically at execution time rather than relying on arguments passed seconds earlier. In React effects, ensure timer callbacks use `useRef` or explicit dependency arrays with proper cleanup.
BUG #8	ERROR RECOVERY	**Network Failure Leaves UI in Permanent Loading State with No Retry** A network drop or 503 gateway timeout occurs. The UI skeleton spinner spins indefinitely, the table is unclickable, and the user must hard-refresh the page to regain control.	**Root Cause:** The catch block in the API call threw an unhandled error, or `state.status = "loading"` was never transitioned to `state.status = "failed"` in `extraReducers.rejected`. **Senior Fix:** 1. Handle `fetchApplications.rejected` in the reducer, setting `state.status = "failed"` and recording `state.error = action.error.message`. 2. Render an error alert component with an explicit "Retry Request" button that dispatches the thunk again.

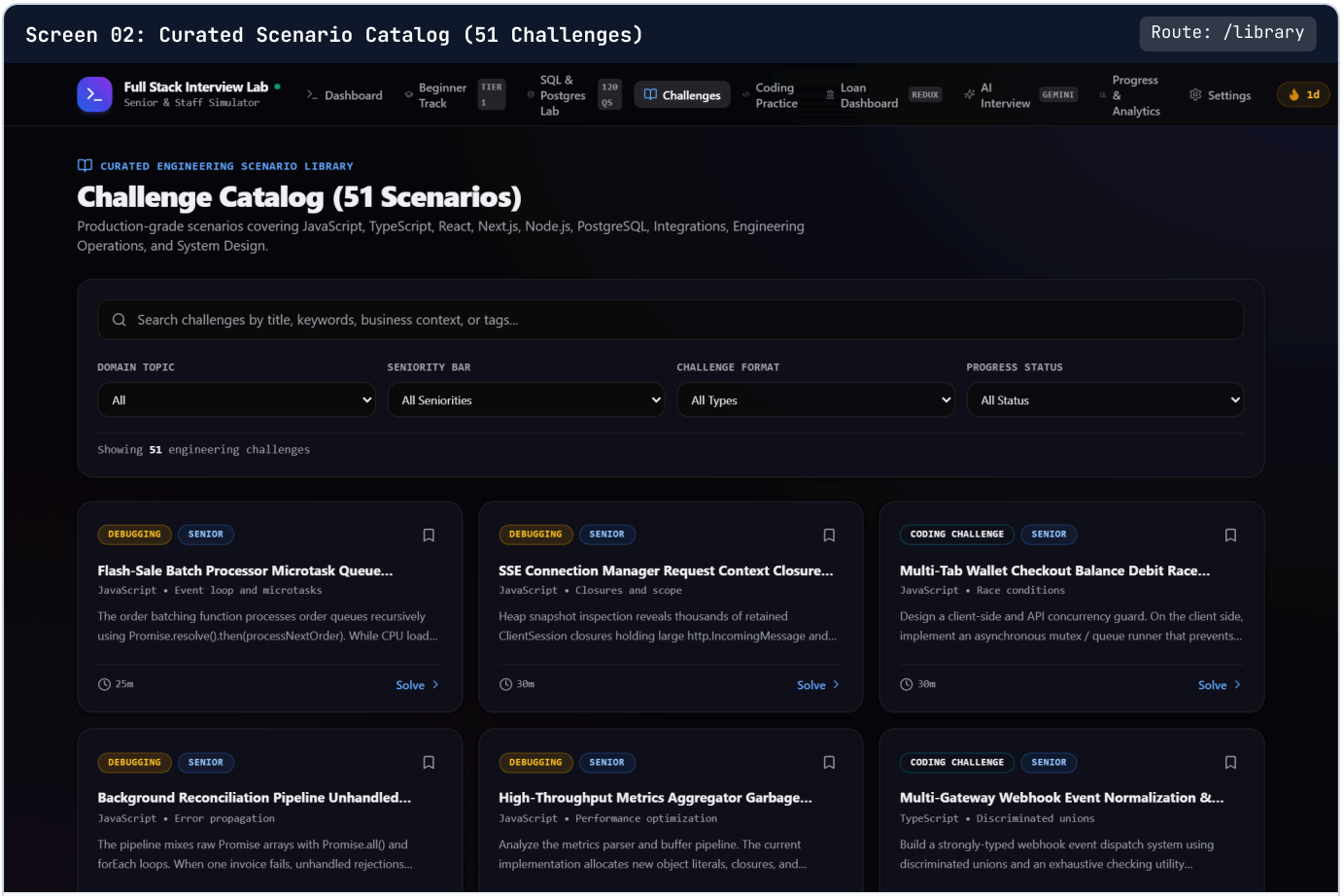
7. Live System Walkthrough & Feature Screenshots

14 SCREENSHOTS

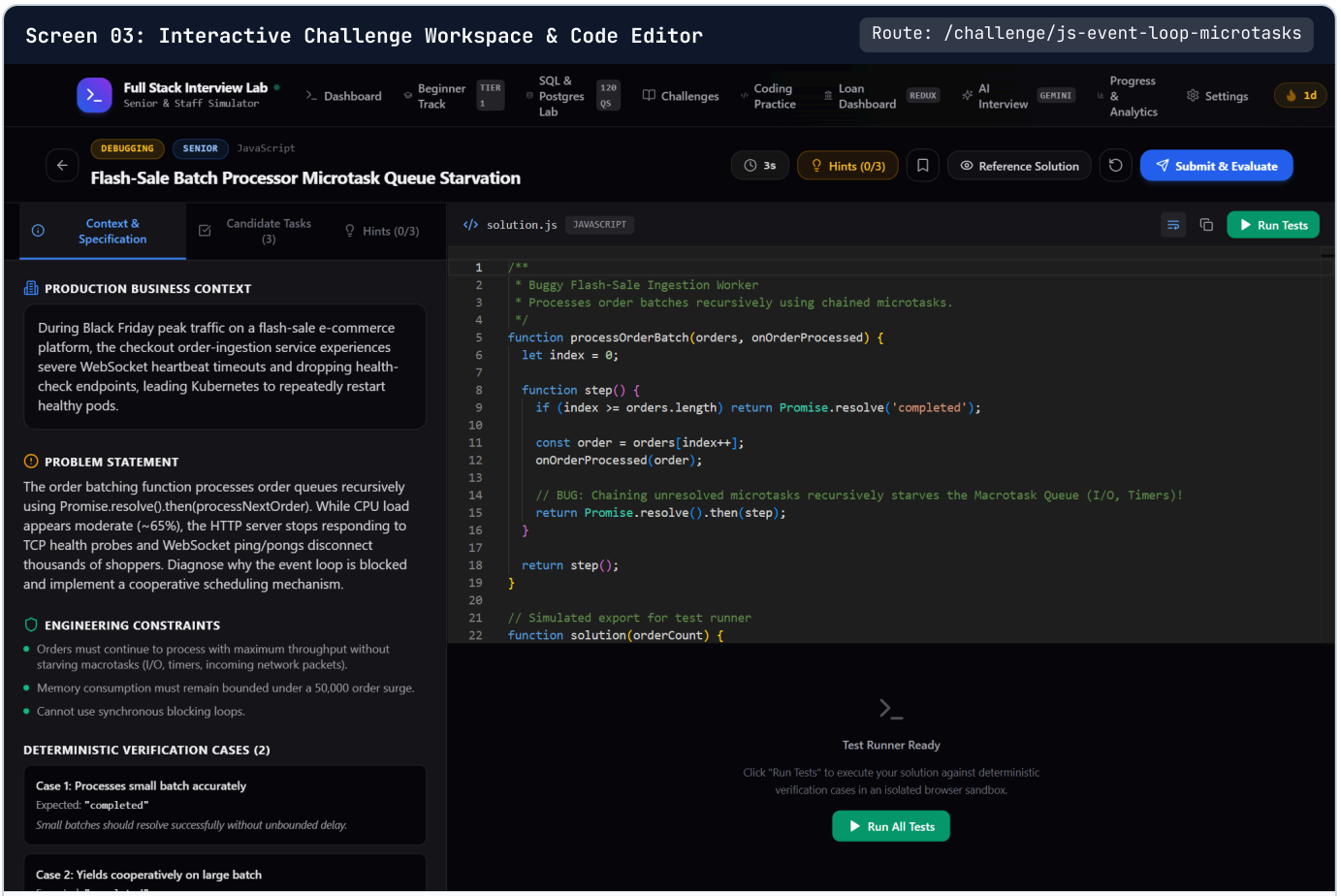
The following high-resolution screenshots are captured directly from the running production instance of the **Full Stack Engineering Interview Lab**, illustrating its modern design system and interactive surfaces.



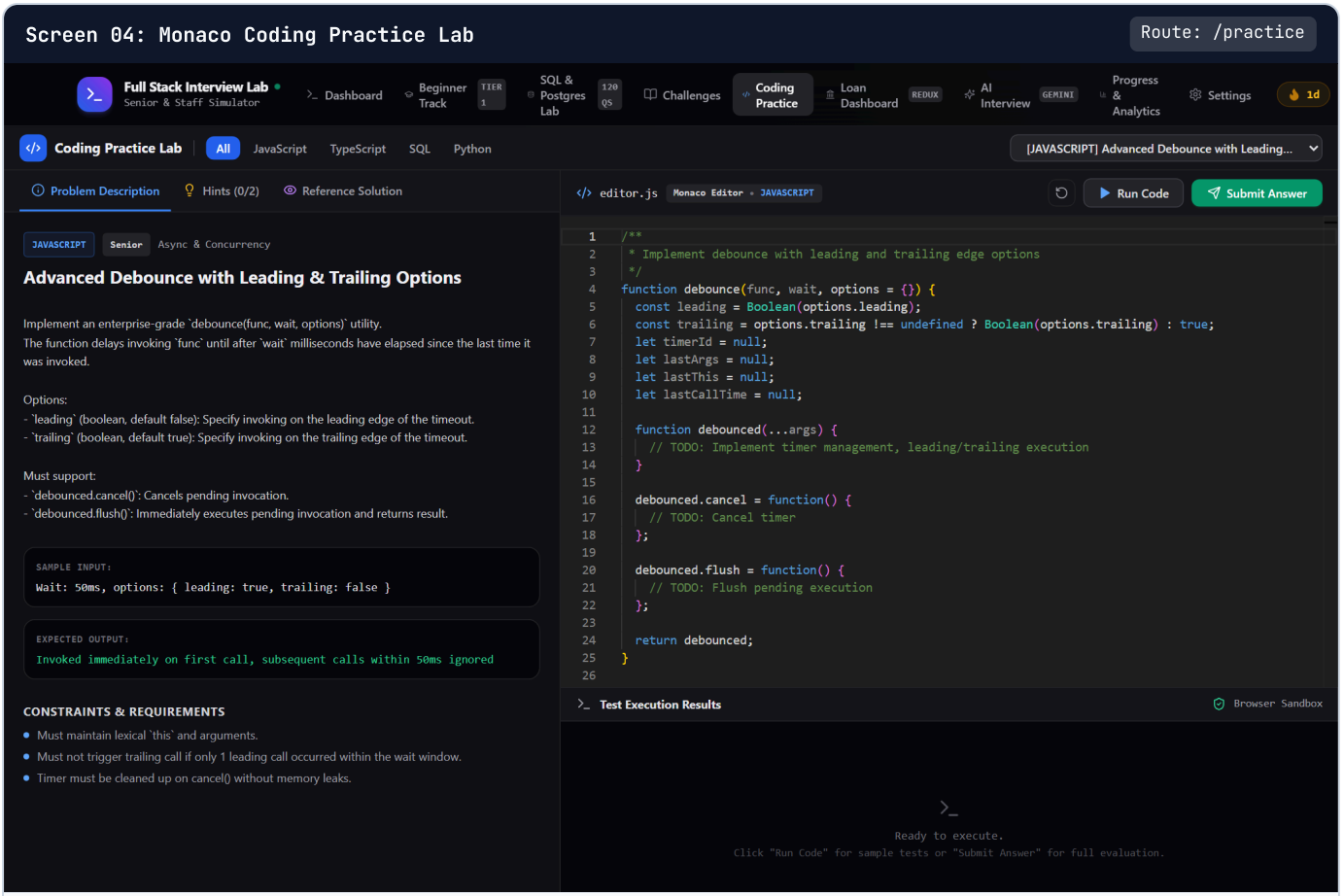
Main Cockpit: Displays the interview readiness radial ring meter, real-time learning streak flame counter, quick action launchpad (including the new SQL & Postgres Lab button), domain mastery progress matrix, and active challenge filter preview.



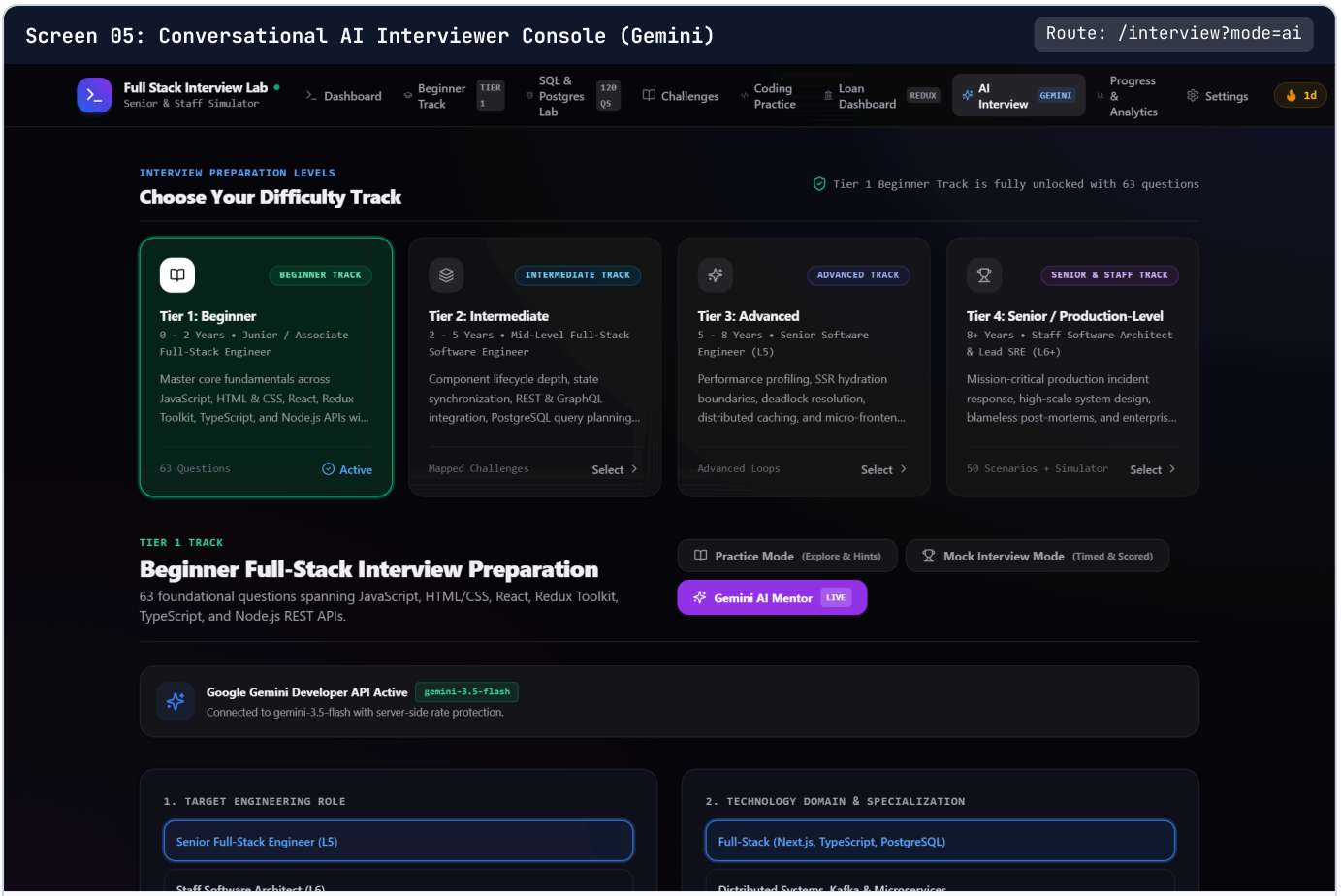
Challenge Library: Live search and multi-dimensional filter bar (Topic, Seniority Bar, Format, Bookmark, and Completion status) with pass-rate badges and time estimates.



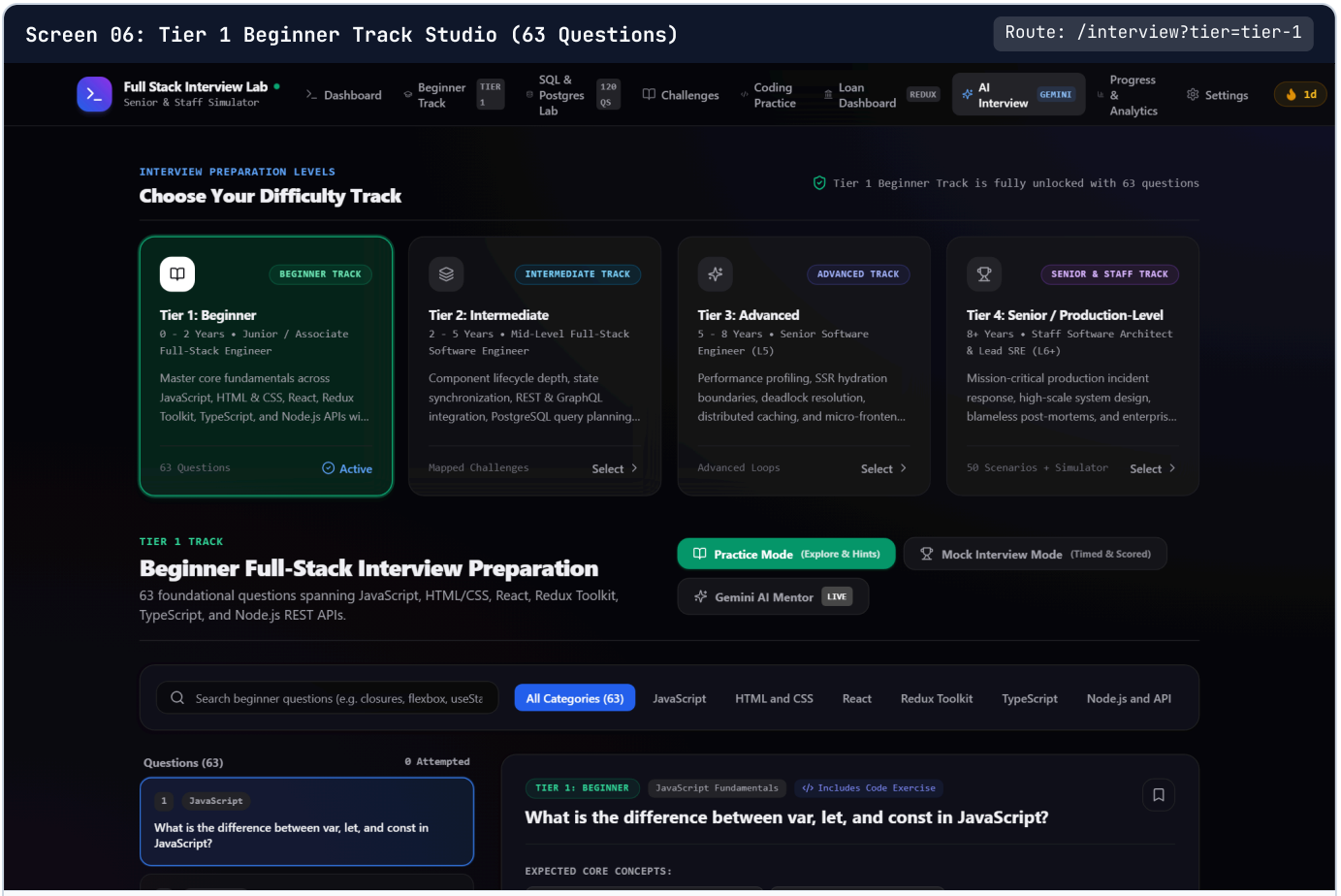
Engineering Workspace: Problem specification panel, production business incident context, telemetry monitoring charts, integrated code editor, progressive hint revelation, and transparent rubric evaluation drawer.



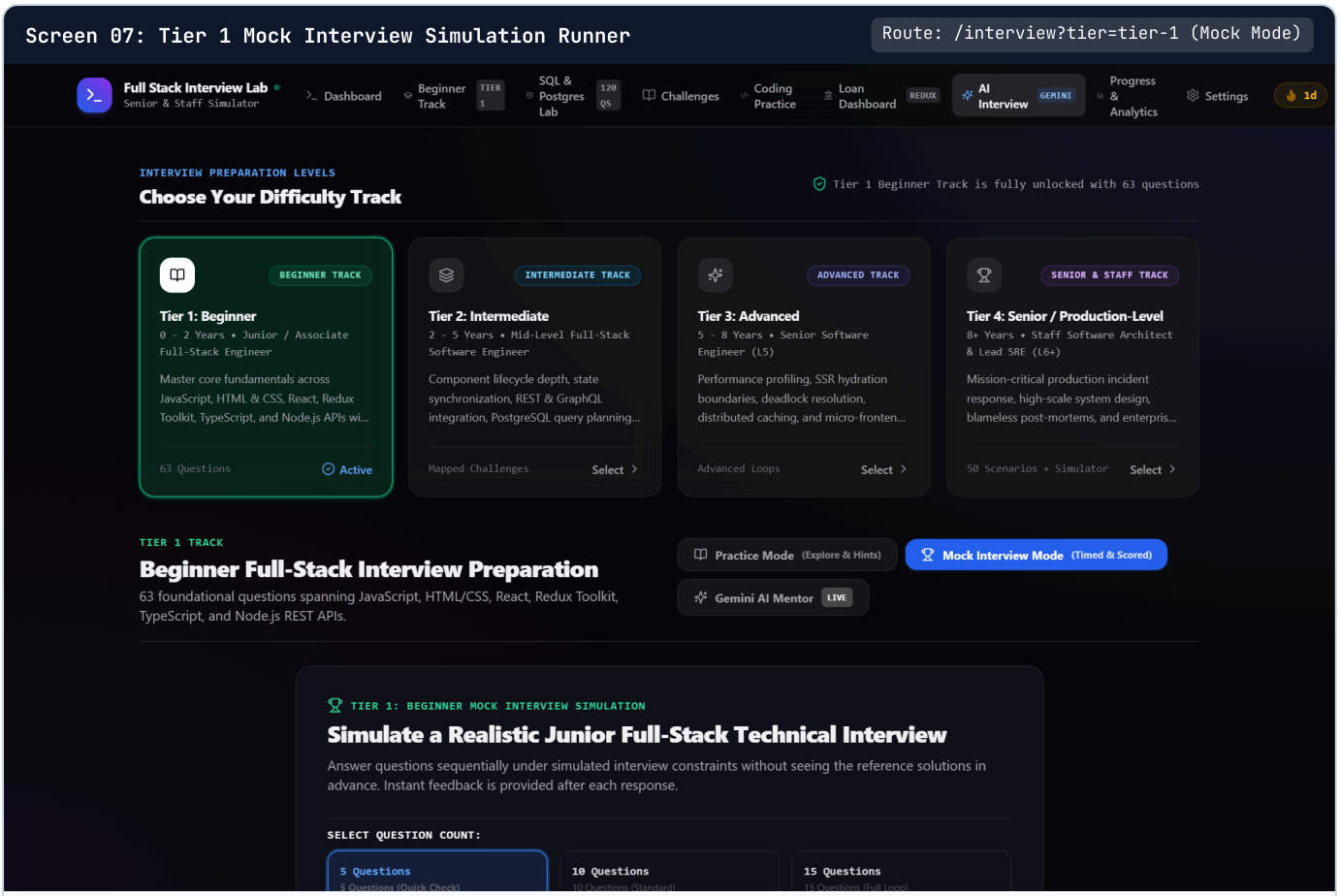
In-Browser Coding Practice: Monaco code editor supporting JavaScript, TypeScript, and SQL with real-time in-browser sandboxed execution, automated test case validation, and output terminal.



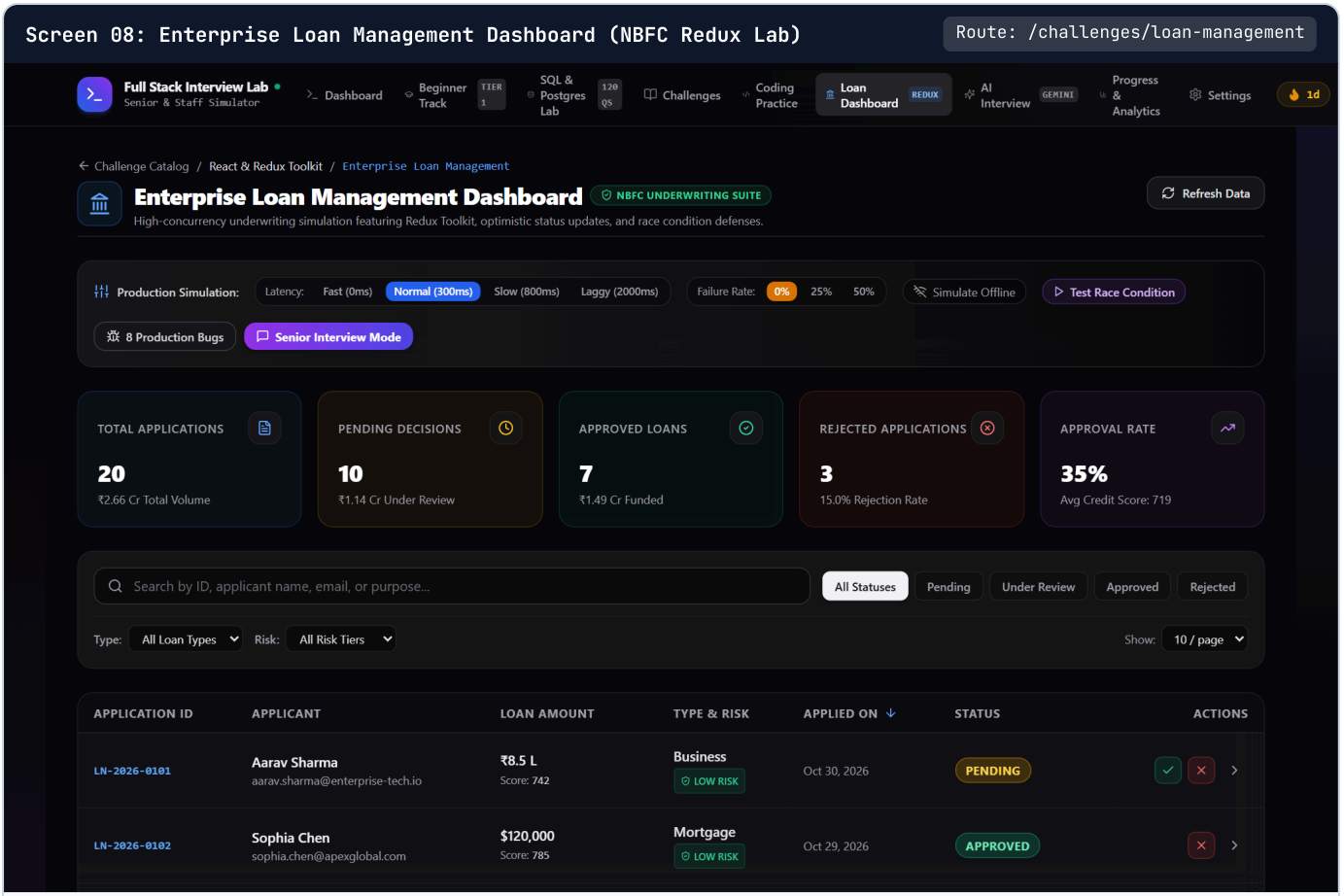
Gemini AI Interviewer: Setup matrix allowing candidates to configure target engineering role (L5–L7), technology domain specialization, hiring committee bar, and turn depth with rate-protected backend execution.



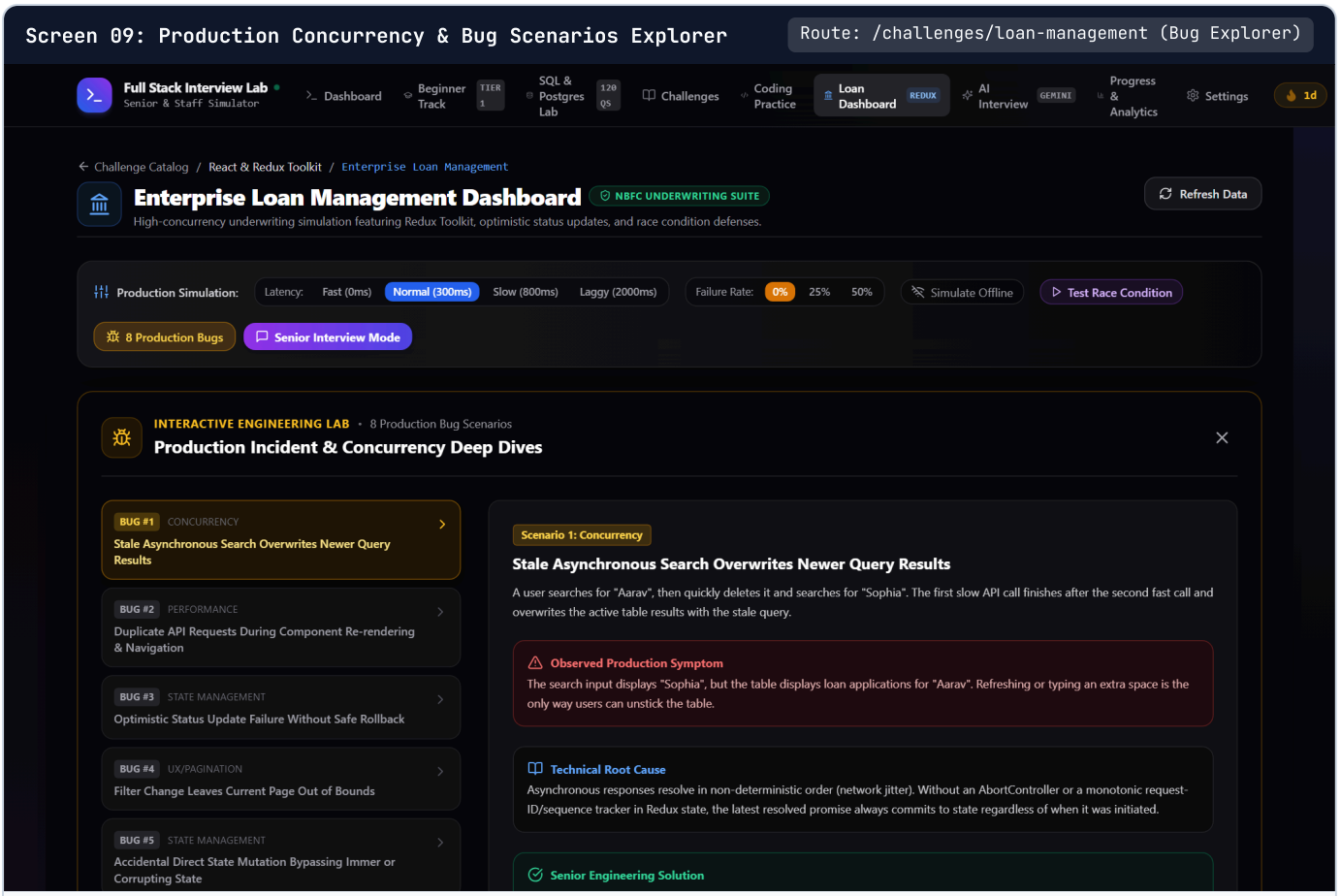
Tier 1 Practice Studio: Question navigator across 6 categories (JavaScript, HTML/CSS, React, Redux Toolkit, TypeScript, Node.js), expected concepts tag matrix, progressive hints, and immediate benchmark evaluation.



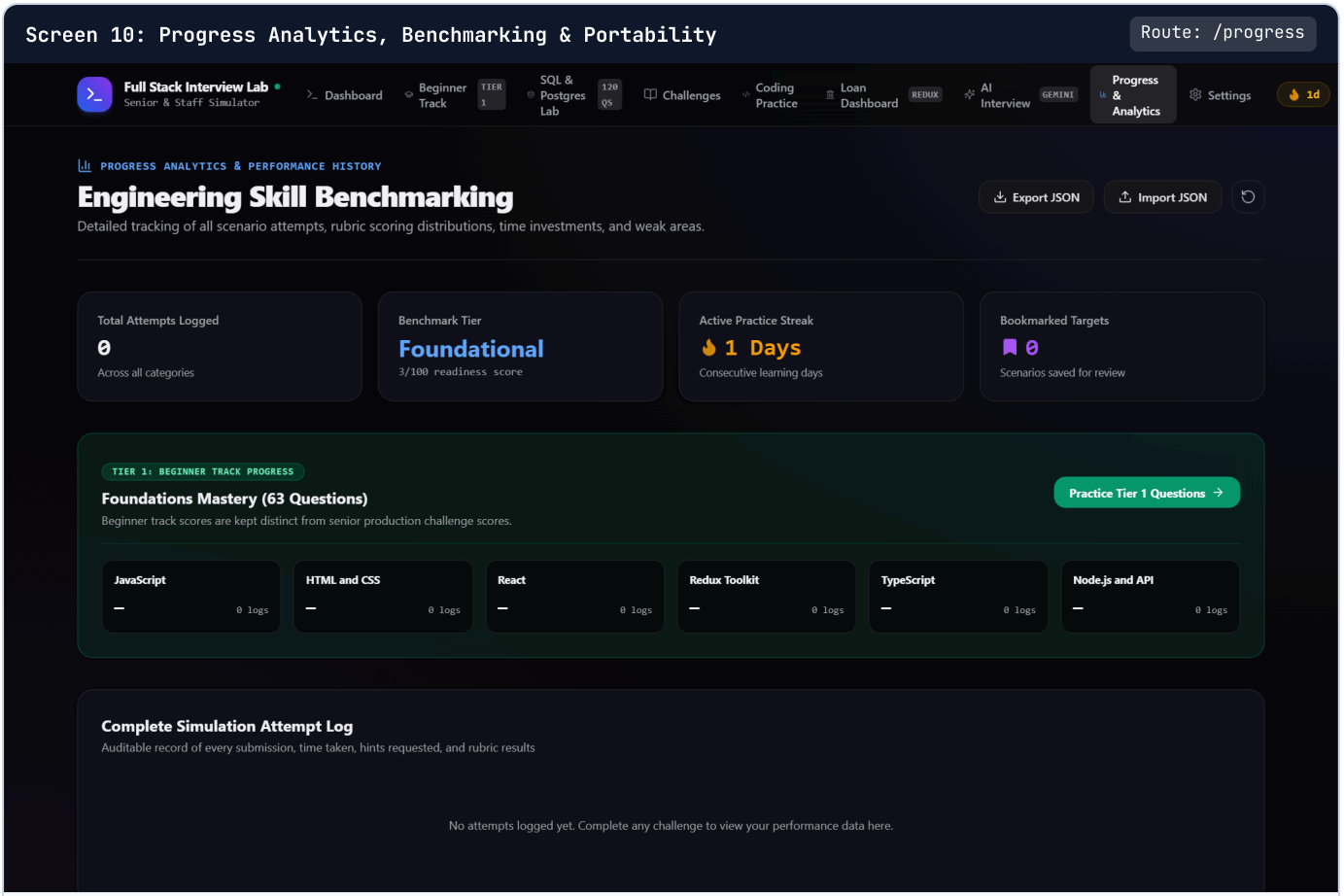
Mock Interview Simulation: Timed sequential interview runner hiding solutions until submission, instant feedback per turn, and comprehensive category performance scorecard upon completion.



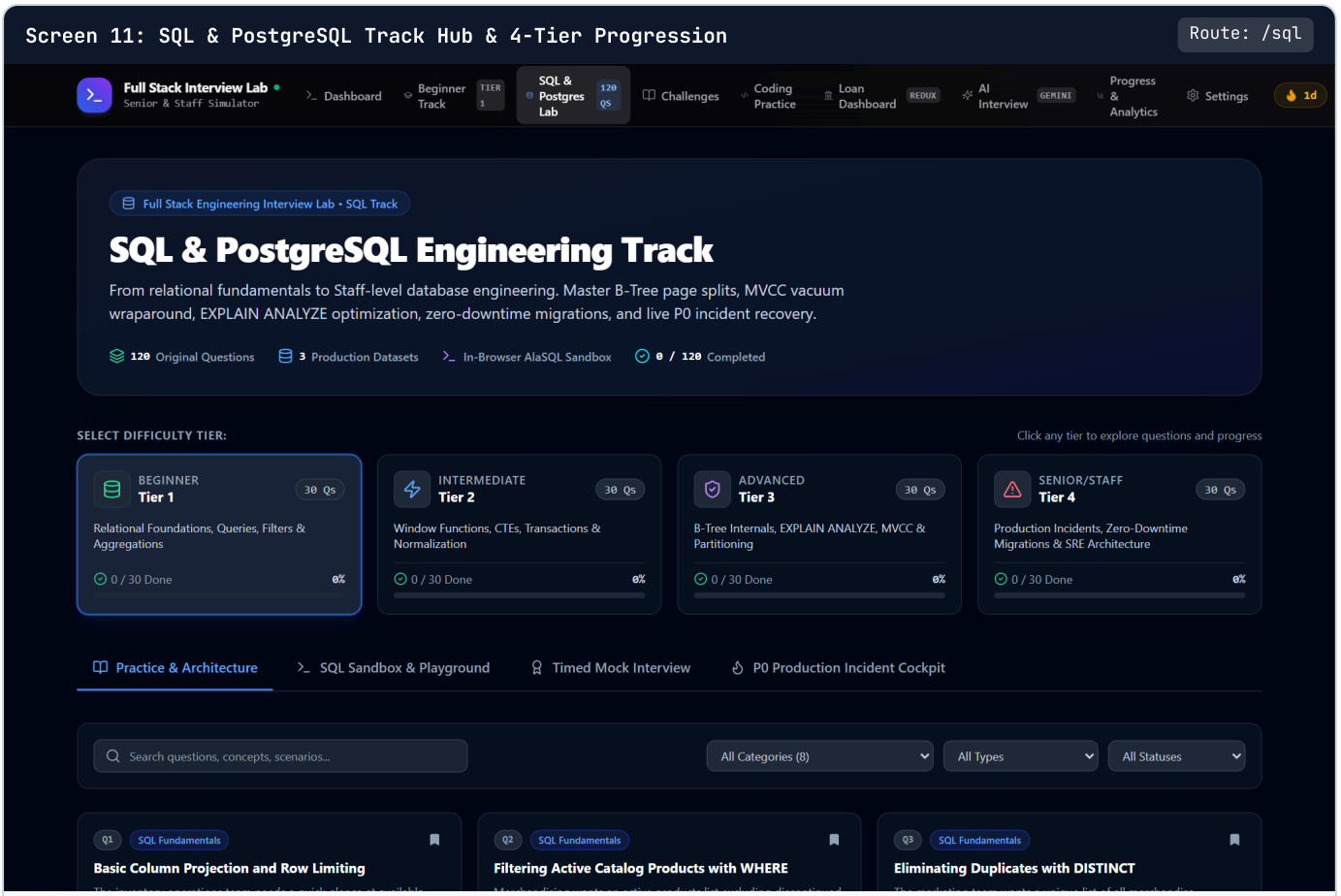
Loan Dashboard: Redux Toolkit state architecture with debounced search, status filter pills, sortable paginated table, optimistic updates, and slide-over loan underwriting details drawer.



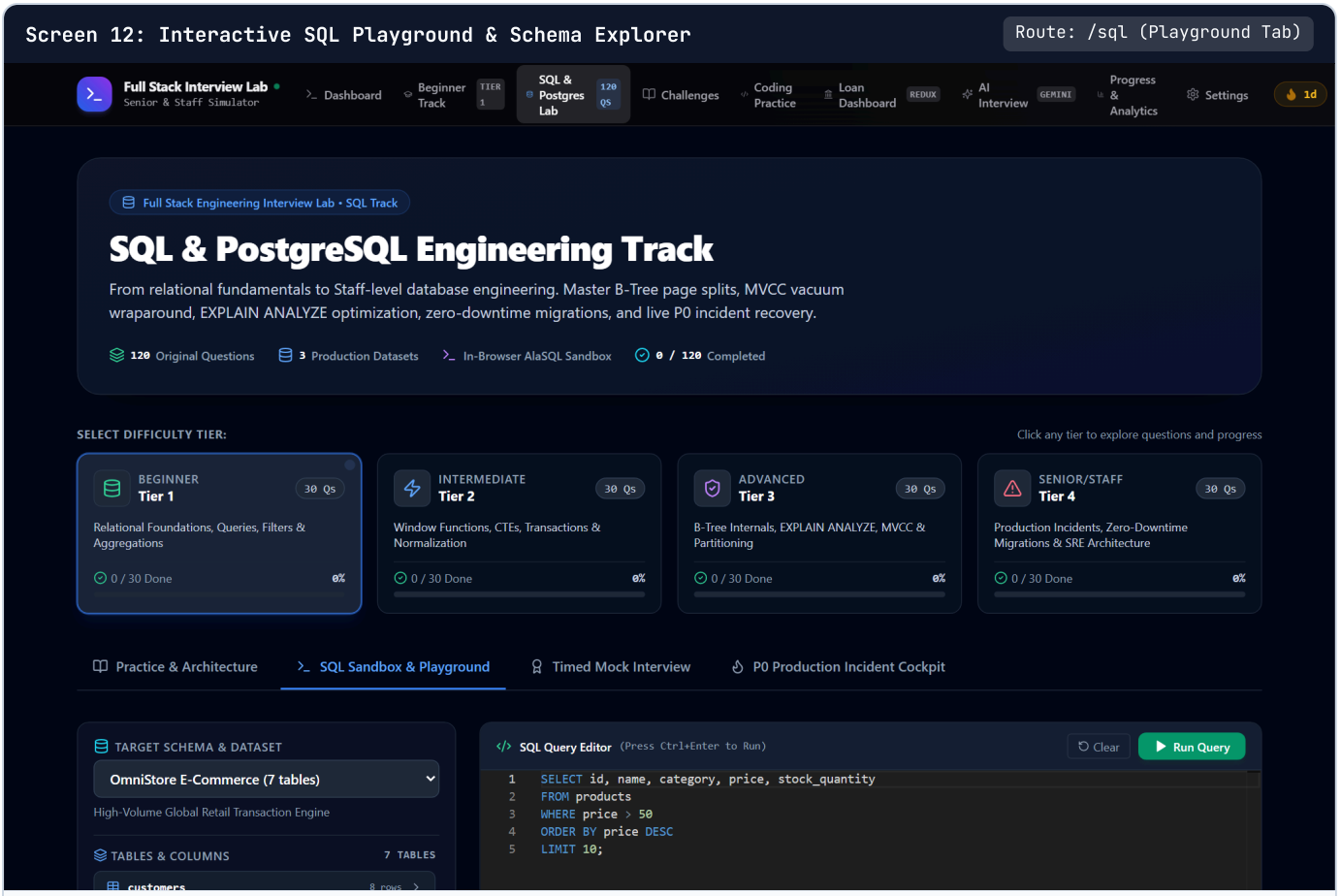
8 Production Bug Scenarios: Interactive explorer covering race conditions, direct Redux state mutation, optimistic update failures without rollback, memory leaks, and network failure recovery.



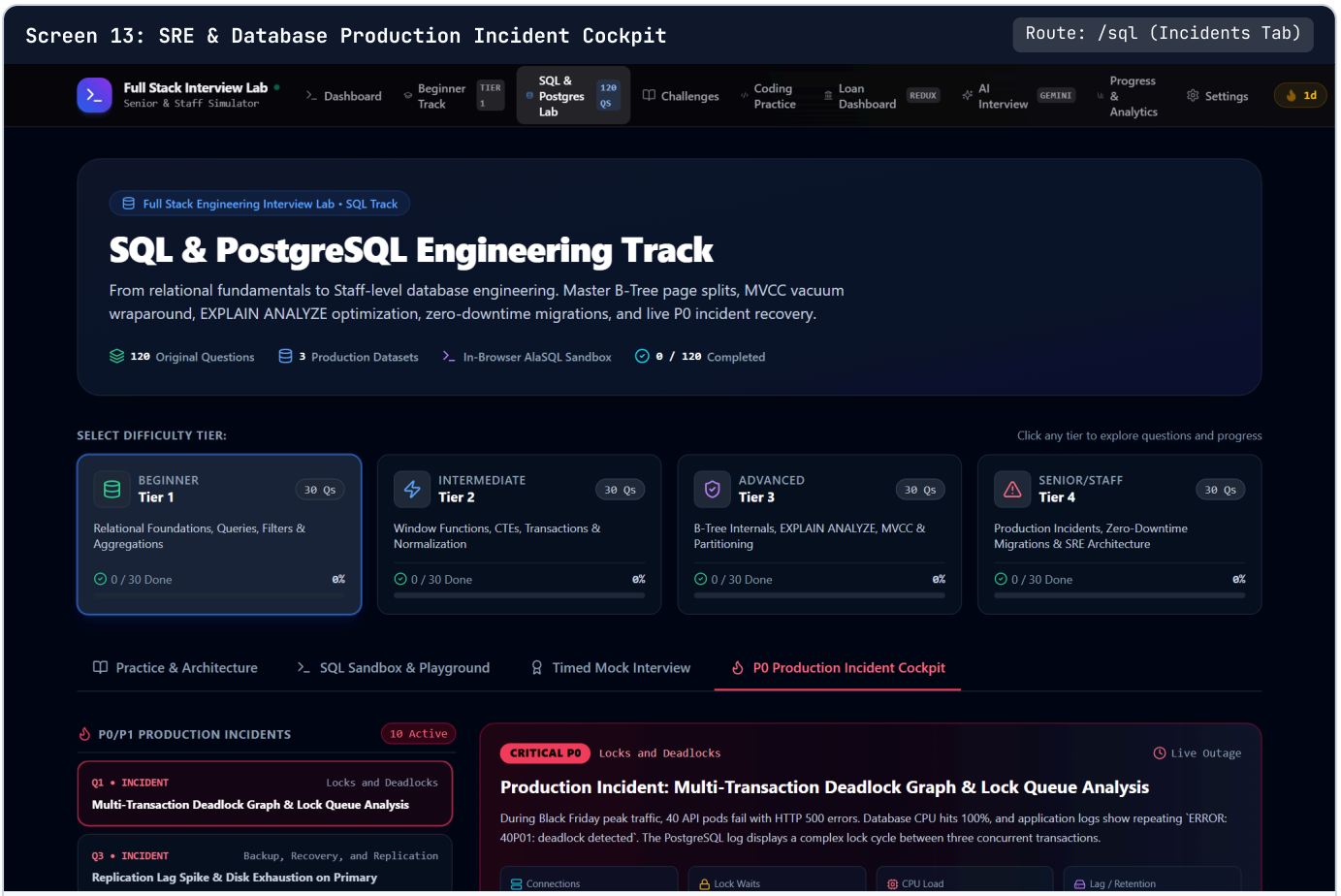
Progress & Analytics: Complete auditable submission history log, benchmark tier rating, streak tracking, Tier 1 category masteries, and one-click JSON backup export/import portability.



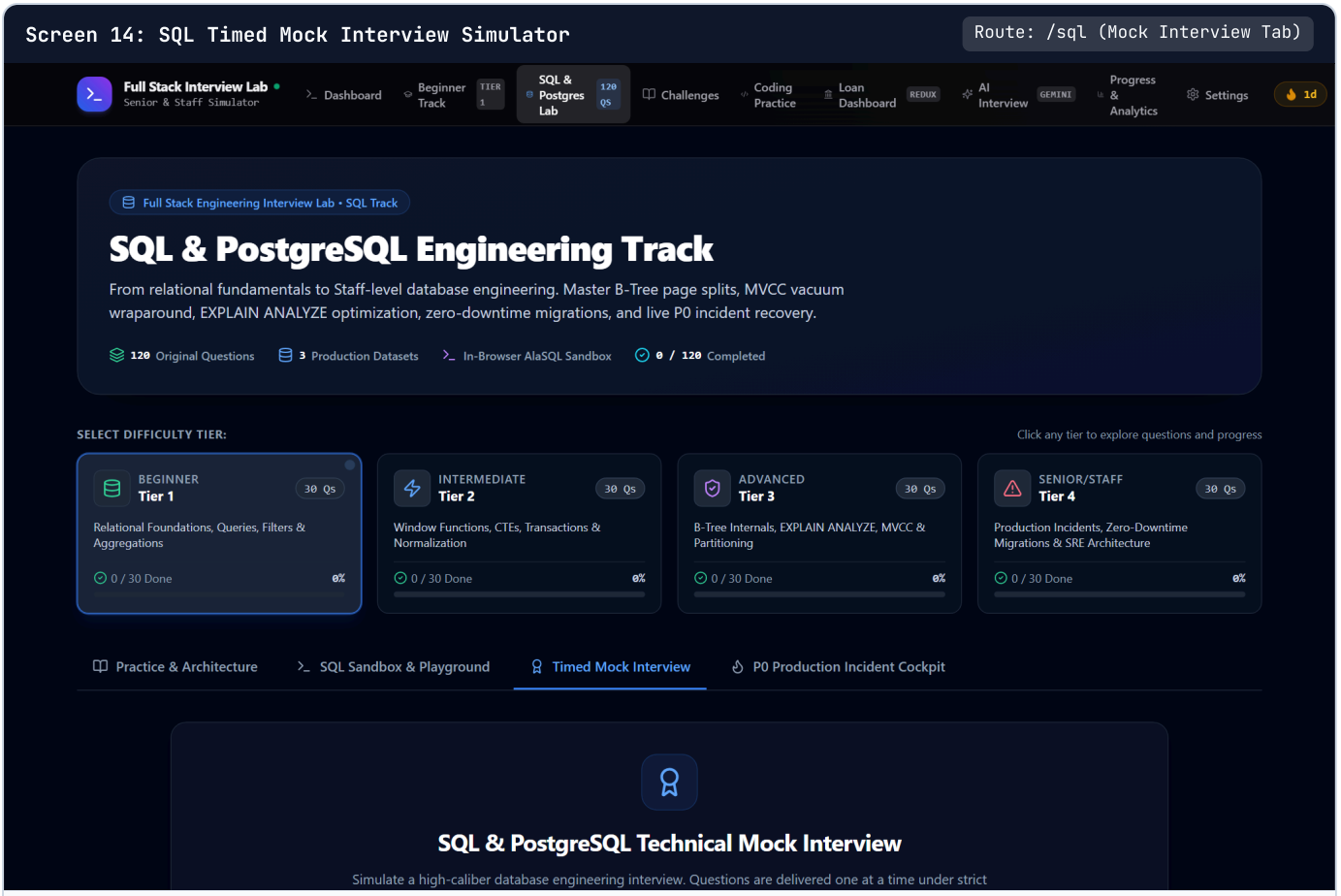
SQL Track Hub: 4 progression tier cards (Tier 1: Beginner SQL, Tier 2: Intermediate SQL, Tier 3: Advanced PostgreSQL, Tier 4: Senior SRE/DBA) with live completion percentage meters, question breakdown counters, and navigation tabs.



SQL Playground: Monaco SQL code editor running queries against in-memory relational datasets (E-commerce, Loan Management, SaaS Analytics), live schema table explorer, instant tabular result viewer, and reset dataset control.



SRE Incident Cockpit: High-severity database outage simulator featuring real-time CPU/IOPS telemetry charts, database server log streams, candidate root-cause diagnostic editor, and Gemini AI post-mortem evaluation.



SQL Mock Interview: Timed sequential database technical interview runner with question count configuration, live countdown timer, solution masking during attempt, and comprehensive evaluation scorecard.